PHÂN HIỆU TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
TẠI TP. HỒ CHÍ MINH

BỘ MÔN CÔNG NGHỆ THÔNG TIN

----ꕥ📖ꕥ----

KHAI PHÁ DỮ LIỆU

BÁO CÁO THỰC HÀNH BUỔI 3

Nguyễn Đức Vinh – 6451071087

CQ.64.CNTT

GVHD: Ths. Nguyễn Thiện Dương

TP. Hồ Chí Minh - 2025

Lưu ý:

1. *Số trang trong tài liệu thực hành tính theo số trang ứng dụng đọc PDF đếm*
2. *SV bố trí mỗi câu ở dưới sẽ là 1 file mã nguồn riêng (file.ipynb)*
3. ***Những câu nào có yêu cầu lập trình viết mã nguồn thì SV phải in ra họ tên – MSSV (lệnh print) trong câu đó***
4. *SV đặt tên cho file mã nguồn mỗi câu như sau:* ***<MSSV>_LAB3_Bai<X>***
5. *Tất cả các câu sau đó sẽ tổng hợp lại và push lên 1 repository duy nhất trên Github*
6. *Đặt tên cho repository theo cú pháp:* ***<MSSV>_Lab3***

BÁO CÁO THỰC HÀNH

SV paste link dẫn đến 1 repo duy nhất của tất cả các câu bên dưới tại dây (lưu ý để ở chế độ public)

https://github.com/Drista4real/6451071087_Lab3.git

Bài 1 (Trang 1) – Phân lớp Iris

Mục tiêu: đọc dữ liệu Iris từ sklearn; khảo sát dữ liệu; chuẩn hóa từng thuộc tính bằng cách chia cho giá trị lớn nhất của cột; chia train/test tỷ lệ 8:2; huấn luyện Decision Tree, Gaussian Naive Bayes, KNN và Neural Network.

Cấu trúc Neural Network: Dense(10, ReLU) -> Dense(20, ReLU) -> Dense(3, Softmax). Mô hình dùng Adam, sparse_categorical_crossentropy, accuracy; huấn luyện 100 epoch với batch_size=1.

Xử lý đánh giá: Accuracy, Precision, Recall, F1-score theo từng lớp và confusion matrix. Ba mẫu mới được chuẩn hóa bằng chính vector max của tập Iris rồi đưa qua cả bốn mô hình.

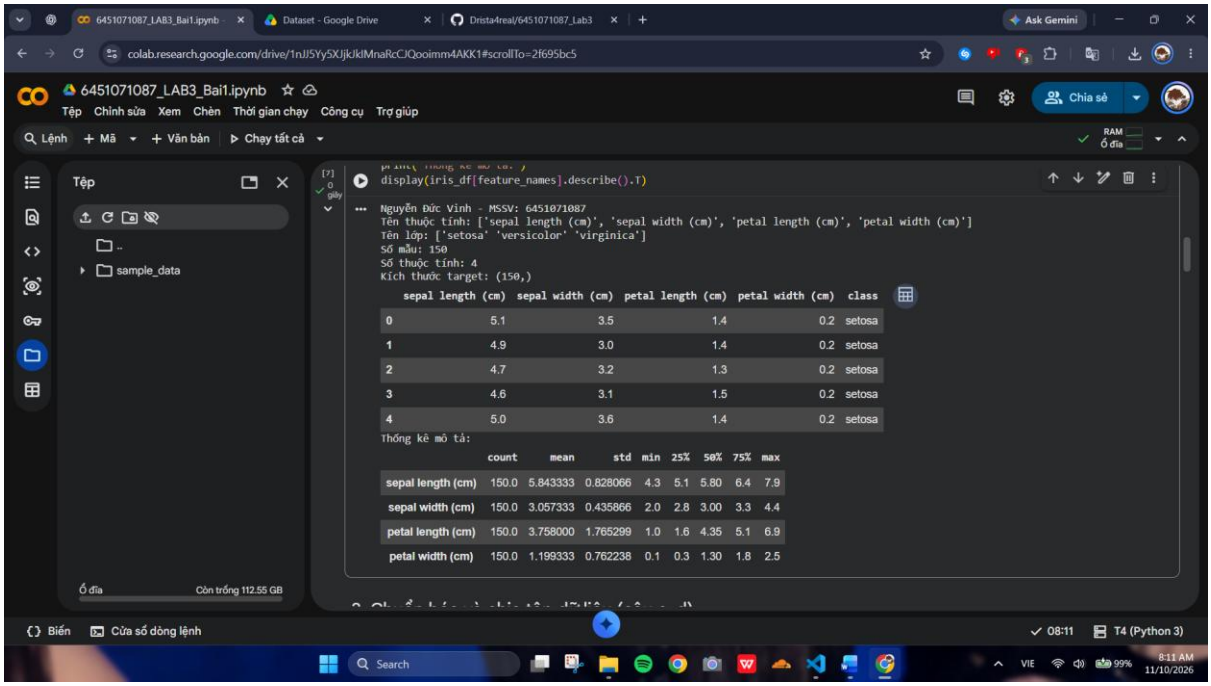
Mã nguồn: Notebook: 6451071087_LAB3_Bai1.ipynb

Đọc và khảo sát dữ liệu Iris (câu a, b):

```
iris = load_iris()
X_raw = iris.data.astype("float32")
y = iris.target
feature_names = iris.feature_names
class_names = iris.target_names

print("Tên thuộc tính:", feature_names)
print("Tên lớp:", class_names)
print("Số mẫu:", X_raw.shape[0])
print("Số thuộc tính:", X_raw.shape[1])
print("Kích thước target:", y.shape)

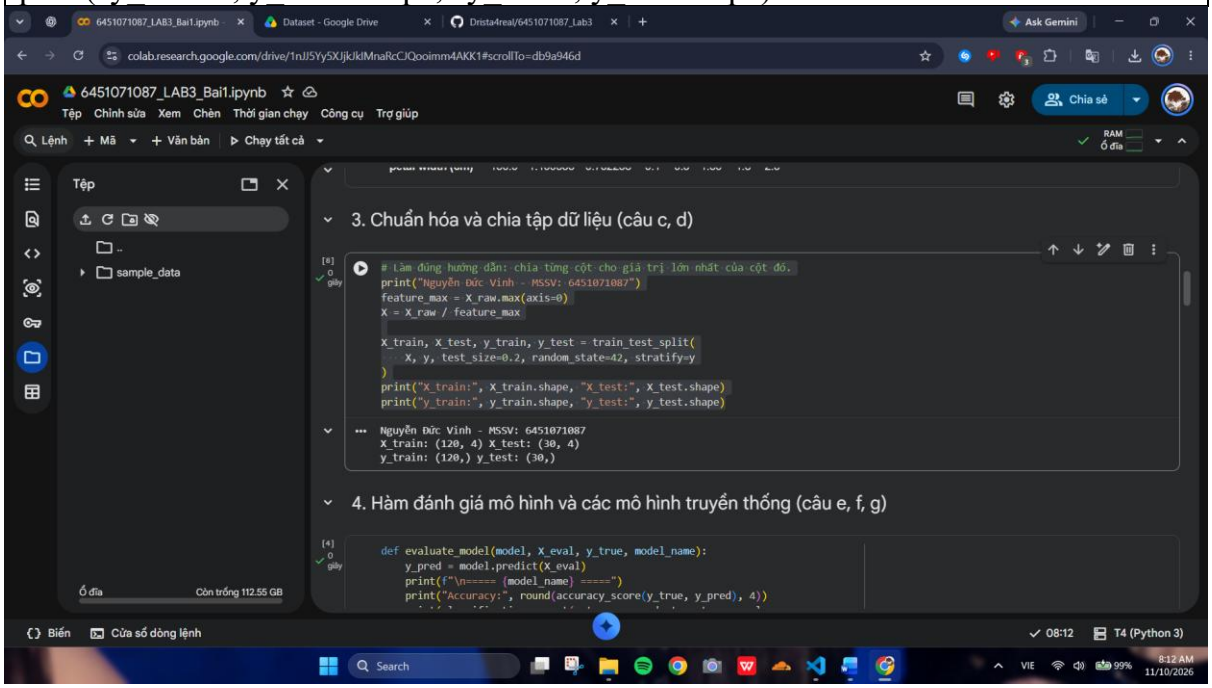
iris_df = pd.DataFrame(X_raw, columns=feature_names)
iris_df["class"] = [class_names[i] for i in y]
display(iris_df.head())
print("Thống kê mô tả:")
display(iris_df[feature_names].describe().T)
```



Chuẩn hóa và chia tập dữ liệu (câu c, d):

```
# Làm đúng hướng dẫn: chia từng cột cho giá trị lớn nhất của cột đó.
print("Nguyễn Đức Vinh - MSSV: 6451071087")
feature_max = X_raw.max(axis=0)
X = X_raw / feature_max

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("X_train:", X_train.shape, "X_test:", X_test.shape)
print("y_train:", y_train.shape, "y_test:", y_test.shape)
```

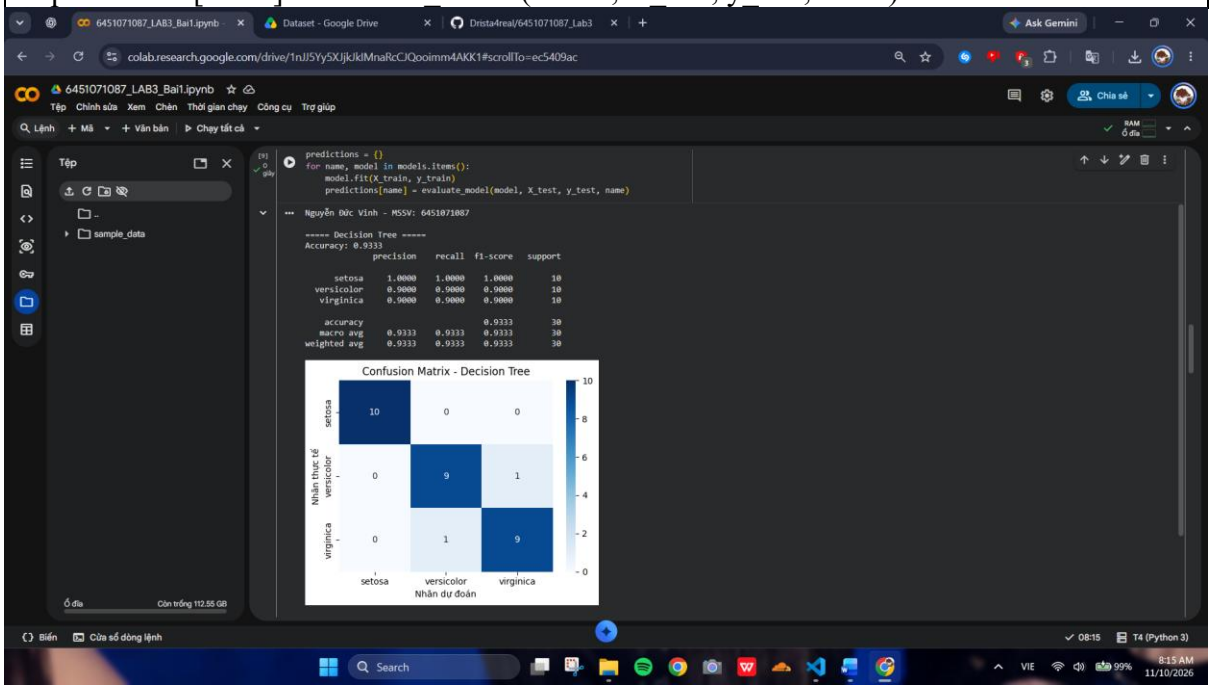


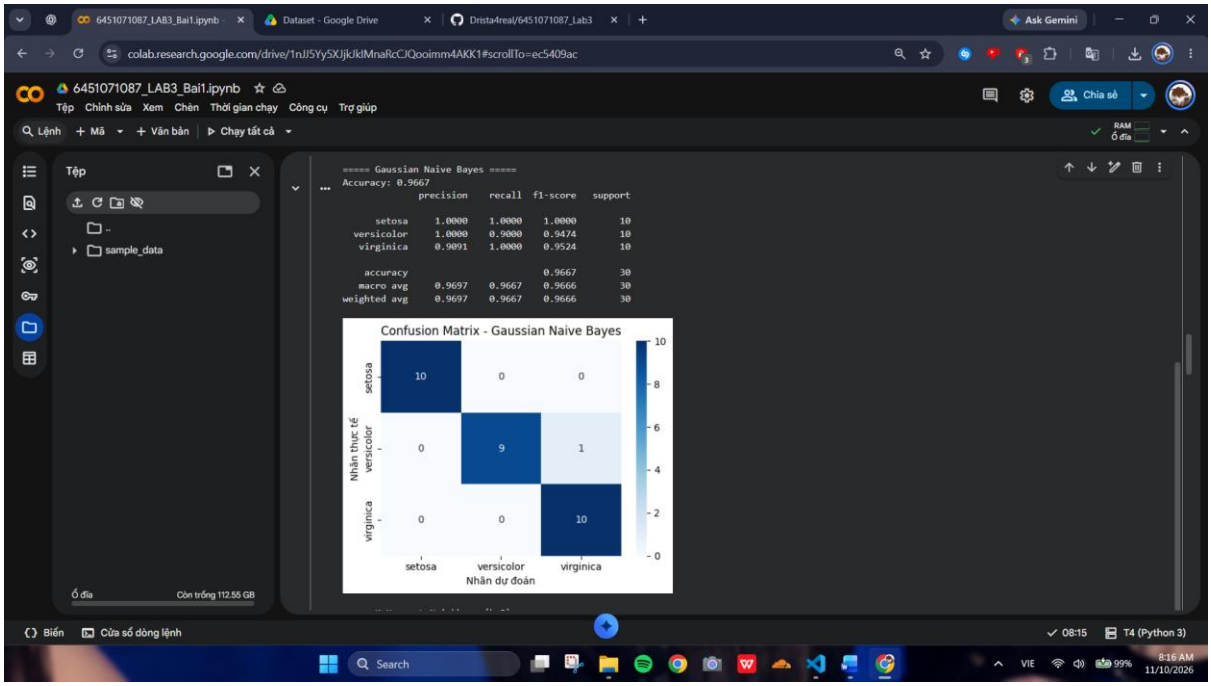
Hàm đánh giá mô hình và các mô hình truyền thống (câu e, f, g):

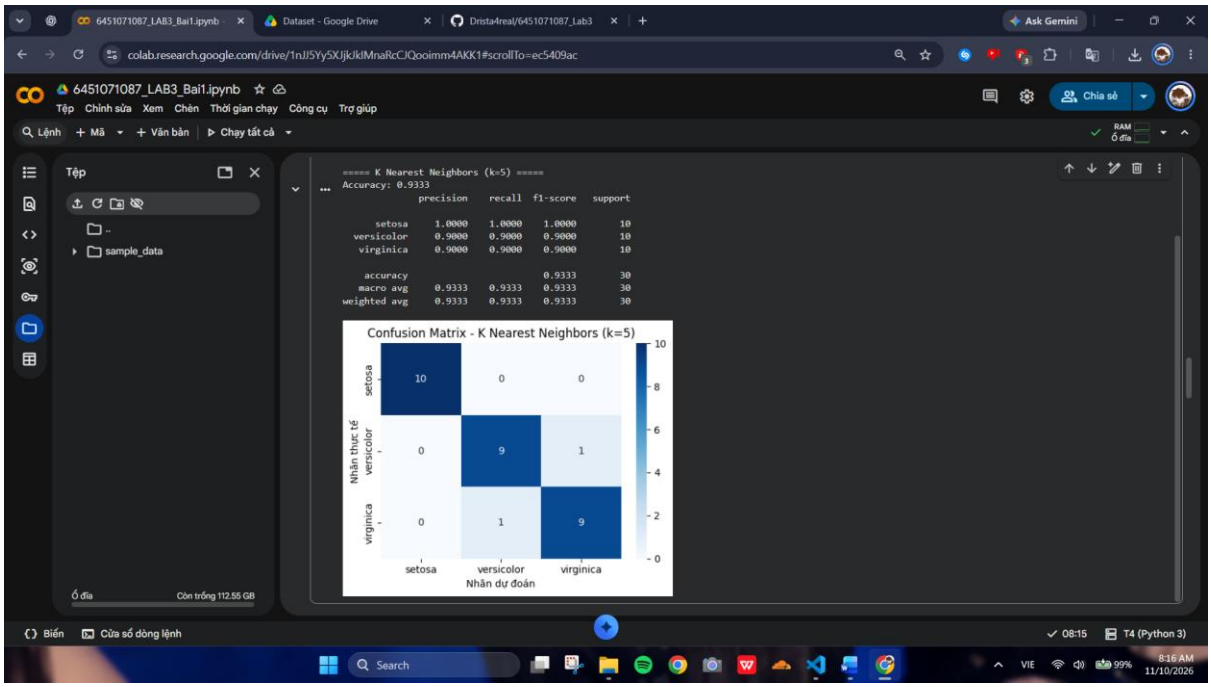
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
def evaluate_model(model, X_eval, y_true, model_name):
    y_pred = model.predict(X_eval)
    print(f"\n===== {model_name} =====")
    print("Accuracy:", round(accuracy_score(y_true, y_pred), 4))
    print(classification_report(y_true, y_pred, target_names=class_names,
                                digits=4, zero_division=0))
    cm = confusion_matrix(y_true, y_pred, labels=np.arange(len(class_names)))
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=class_names, yticklabels=class_names)
    plt.xlabel("Nhãn dự đoán")
    plt.ylabel("Nhãn thực tế")
    plt.title(f"Confusion Matrix - {model_name}")
    plt.tight_layout()
    plt.show()
    return y_pred

models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Gaussian Naive Bayes": GaussianNB(),
    "K Nearest Neighbors (k=5)": KNeighborsClassifier(n_neighbors=5)
}

predictions = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions[name] = evaluate_model(model, X_test, y_test, name)
```







Neural Network bằng Keras (câu h):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
def evaluate_model(model, X_eval, y_true, model_name):
    y_pred = model.predict(X_eval)
    print(f"\n===== {model_name} =====")
    print("Accuracy:", round(accuracy_score(y_true, y_pred), 4))
    print(classification_report(y_true, y_pred, target_names=class_names,
                                digits=4, zero_division=0))
    cm = confusion_matrix(y_true, y_pred, labels=np.arange(len(class_names)))
    plt.figure(figsize=(5, 4))
```

```
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=class_names, yticklabels=class_names)
    plt.xlabel("Nhãn dự đoán")
    plt.ylabel("Nhãn thực tế")
    plt.title(f"Confusion Matrix - {model_name}")
    plt.tight_layout()
    plt.show()
    return y_pred

models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Gaussian Naive Bayes": GaussianNB(),
    "K Nearest Neighbors (k=5)": KNeighborsClassifier(n_neighbors=5)
}

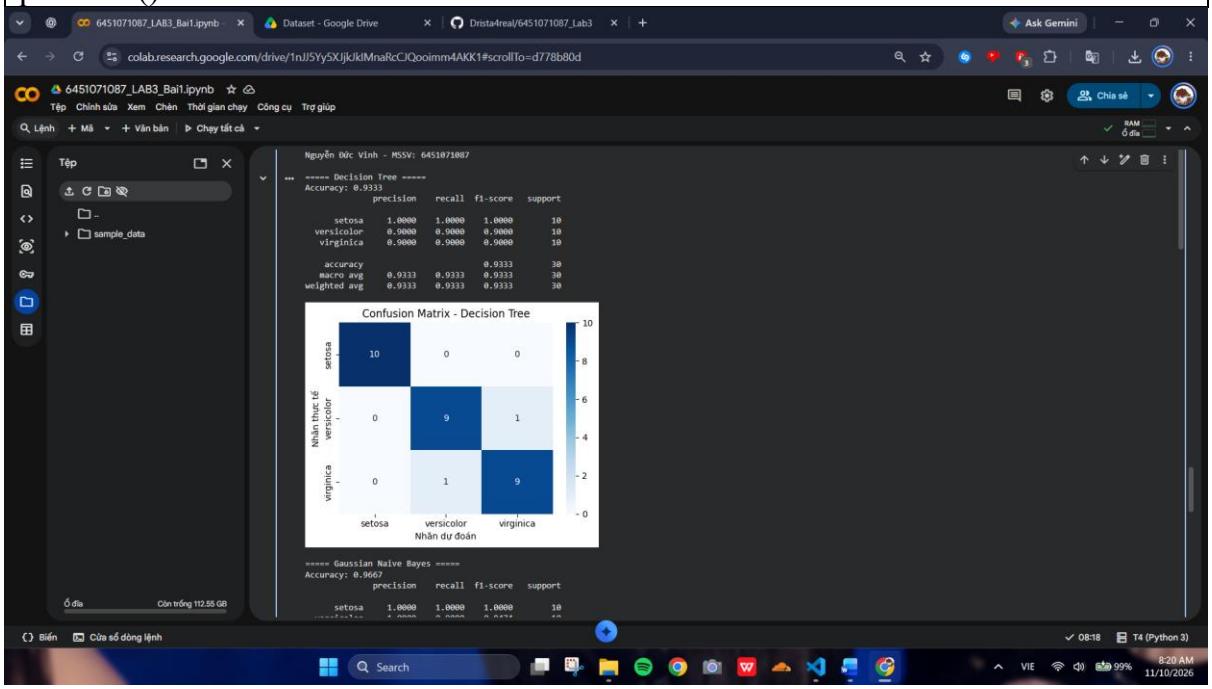
predictions = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions[name] = evaluate_model(model, X_test, y_test, name)
nn_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(10, activation="relu"),
    tf.keras.layers.Dense(20, activation="relu"),
    tf.keras.layers.Dense(len(class_names), activation="softmax")
])

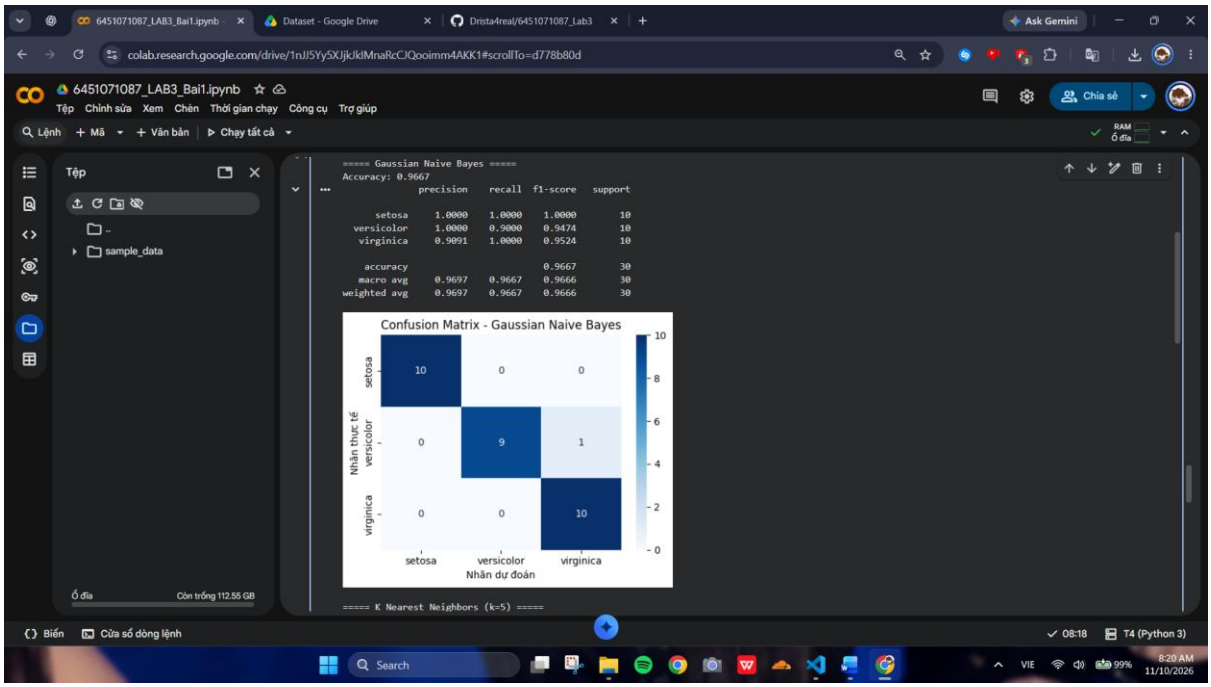
nn_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

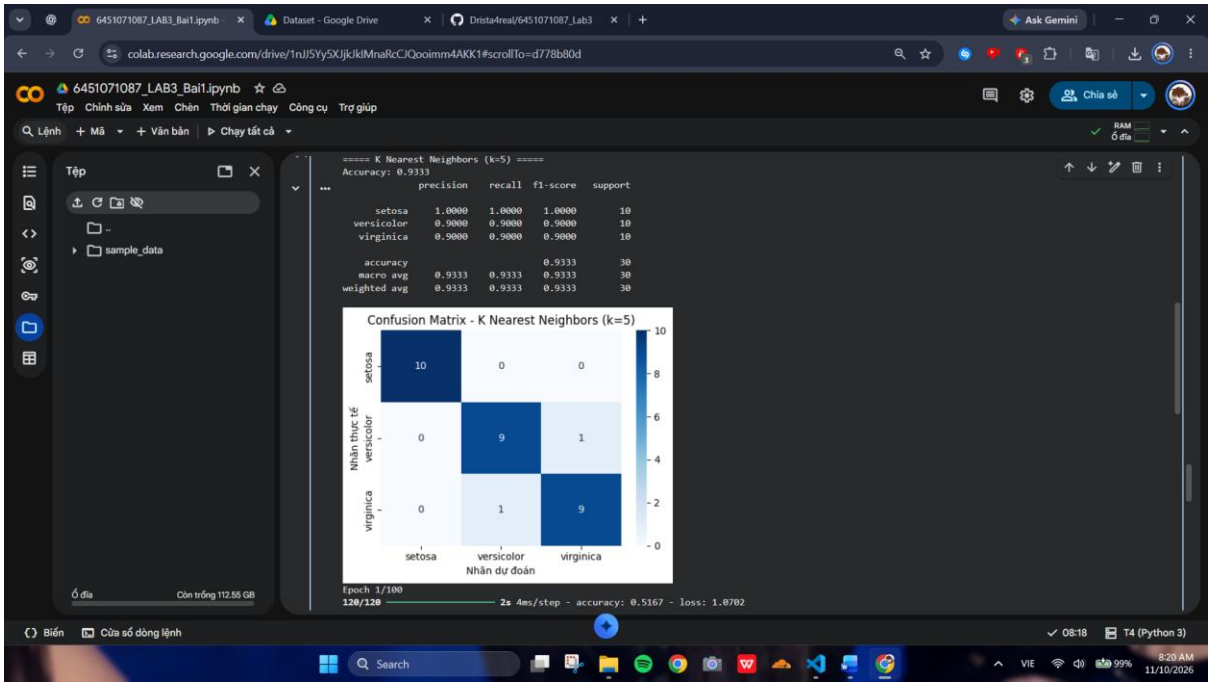
history = nn_model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=1,
    verbose=1
)

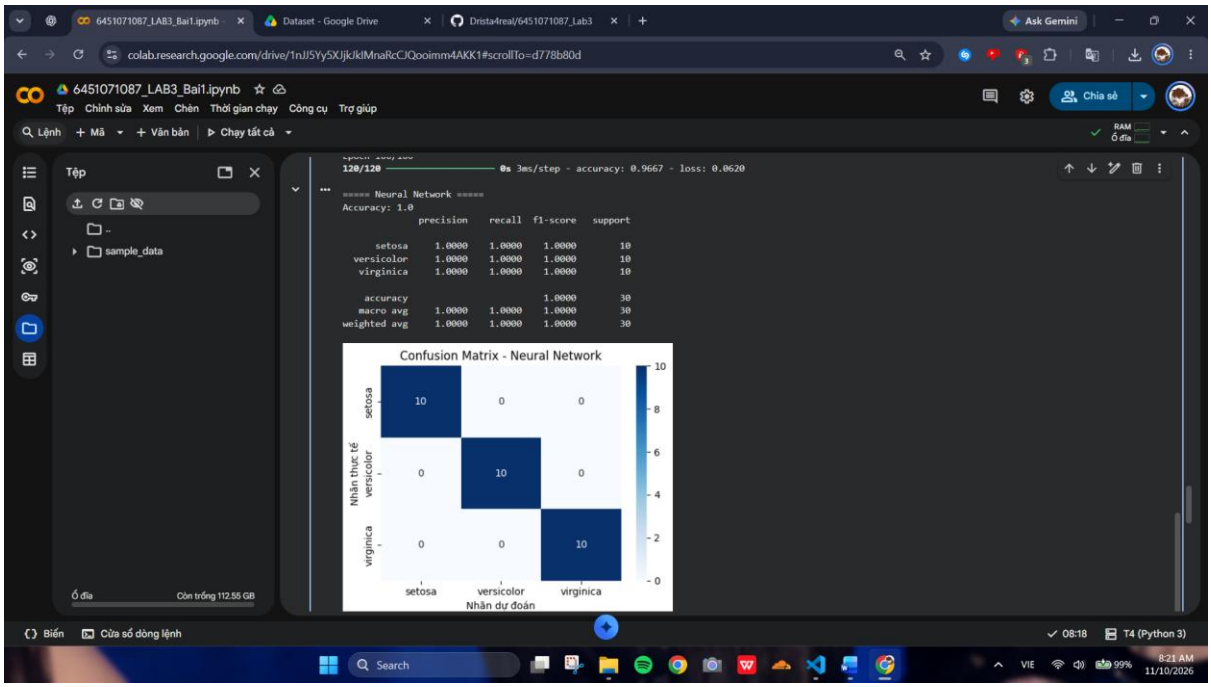
nn_prob = nn_model.predict(X_test, verbose=0)
nn_pred = np.argmax(nn_prob, axis=1)
predictions["Neural Network"] = nn_pred
print("\n===== Neural Network =====")
print("Accuracy:", round(accuracy_score(y_test, nn_pred), 4))
print(classification_report(y_test, nn_pred, target_names=class_names,
                            digits=4, zero_division=0))
```

```
cm = confusion_matrix(y_test, nn_pred, labels=np.arange(len(class_names)))
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Nhãn dự đoán")
plt.ylabel("Nhãn thực tế")
plt.title("Confusion Matrix - Neural Network")
plt.tight_layout()
plt.show()
```







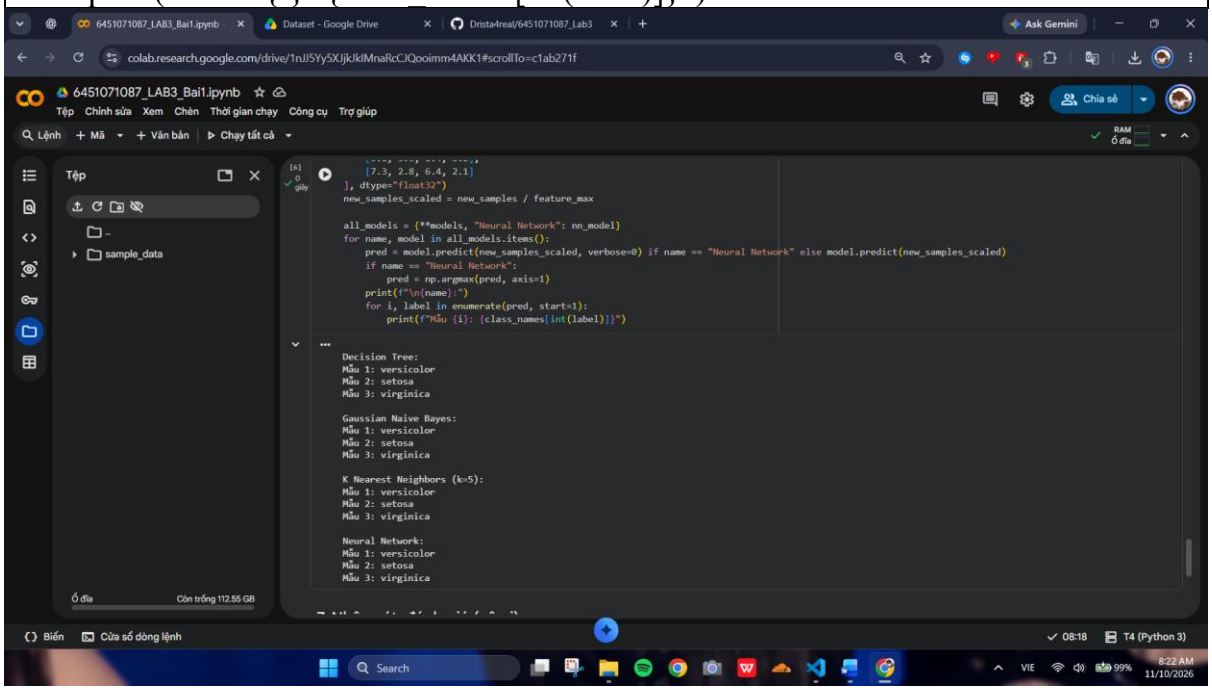


Dự đoán ba mẫu dữ liệu mới (câu i):

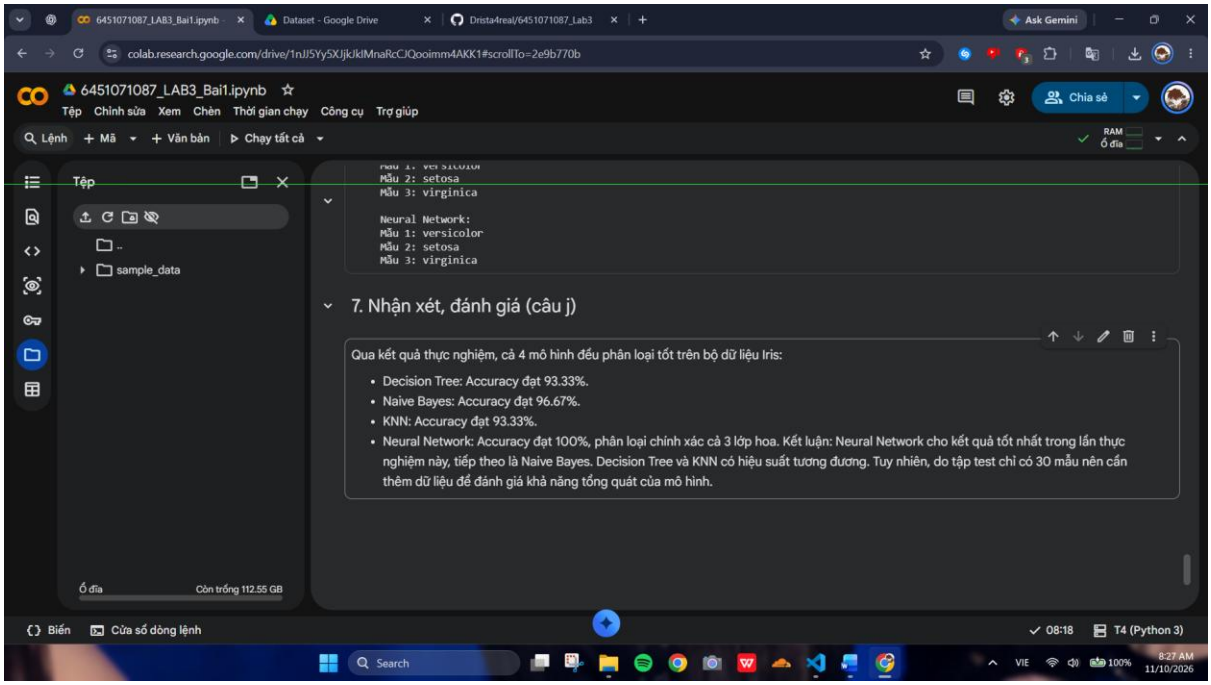
```
new_samples = np.array([
    [6.2, 2.9, 4.3, 1.3],
    [5.1, 3.5, 1.4, 0.2],
    [7.3, 2.8, 6.4, 2.1]
], dtype="float32")
new_samples_scaled = new_samples / feature_max

all_models = {**models, "Neural Network": nn_model}
for name, model in all_models.items():
```

```
    pred = model.predict(new_samples_scaled, verbose=0) if name == "Neural
Network" else model.predict(new_samples_scaled)
    if name == "Neural Network":
        pred = np.argmax(pred, axis=1)
    print(f"\n{name}:")
    for i, label in enumerate(pred, start=1):
        print(f"Mẫu {i}: {class_names[int(label)]}")
```



Nhận xét, đánh giá (câu j):



Bài 2 (Trang 5) – Neural Network trên MNIST

Dữ liệu MNIST gồm ảnh grayscale 28x28. Tập train và test được nạp bằng tf.keras.datasets.mnist.load_data(); pixel được chia cho 255.0. Tập train ban đầu được chia thành train/validation theo tỷ lệ 8:2.

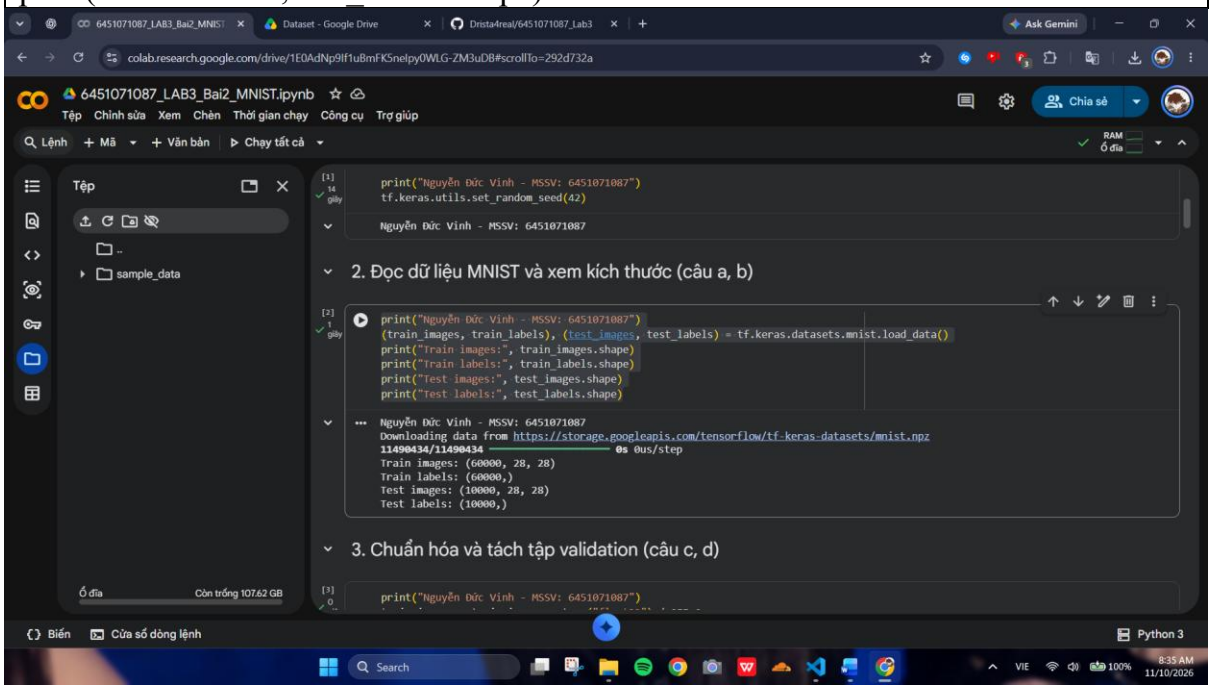
Mô hình: Flatten -> Dense(128, ReLU) -> Dropout(0.2) -> Dense(10, Softmax). Cấu hình compile: Adam, sparse_categorical_crossentropy, accuracy. Huấn luyện 5 epoch, batch_size=32.

Đánh giá trên test gồm Accuracy, classification report, confusion matrix; vẽ accuracy/loss theo epoch; hiển thị 5 ảnh đầu và 5 ảnh dự đoán sai đầu tiên.

Mã nguồn: Notebook: 6451071087_LAB3_Bai2_MNIST.ipynb

Đọc dữ liệu MNIST và xem kích thước (câu a, b):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()
print("Train images:", train_images.shape)
print("Train labels:", train_labels.shape)
print("Test images:", test_images.shape)
print("Test labels:", test_labels.shape)
```

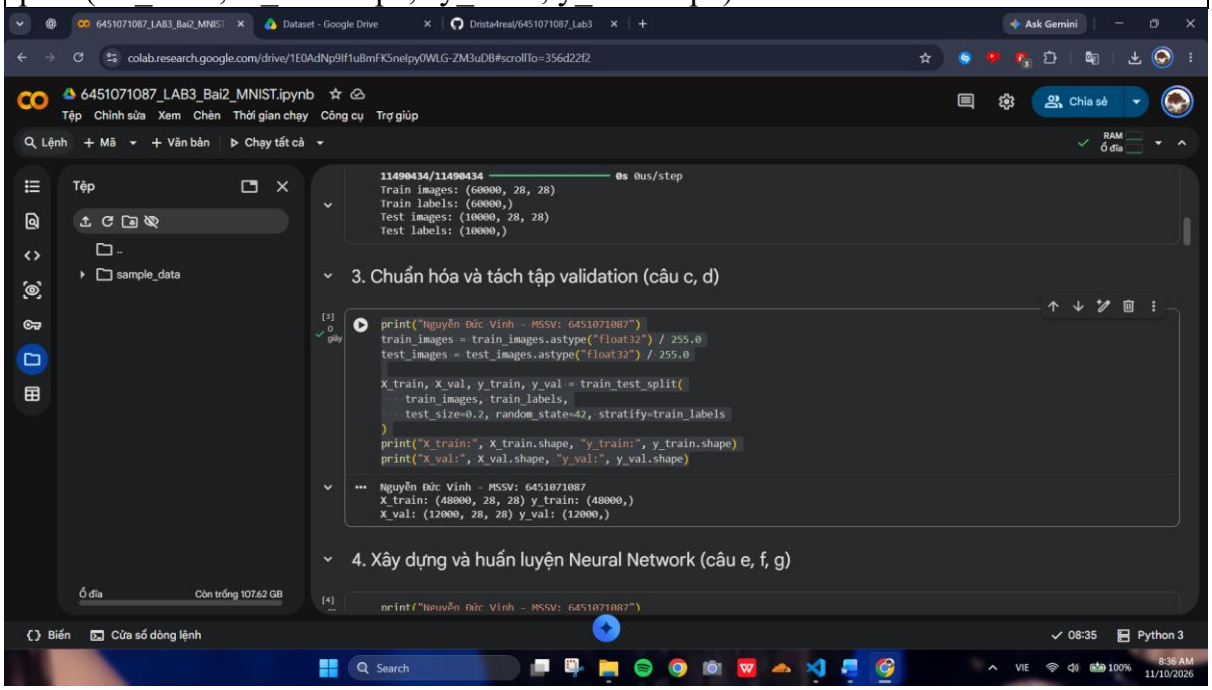


Chuẩn hóa và tách tập validation (câu c, d):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
train_images = train_images.astype("float32") / 255.0
test_images = test_images.astype("float32") / 255.0

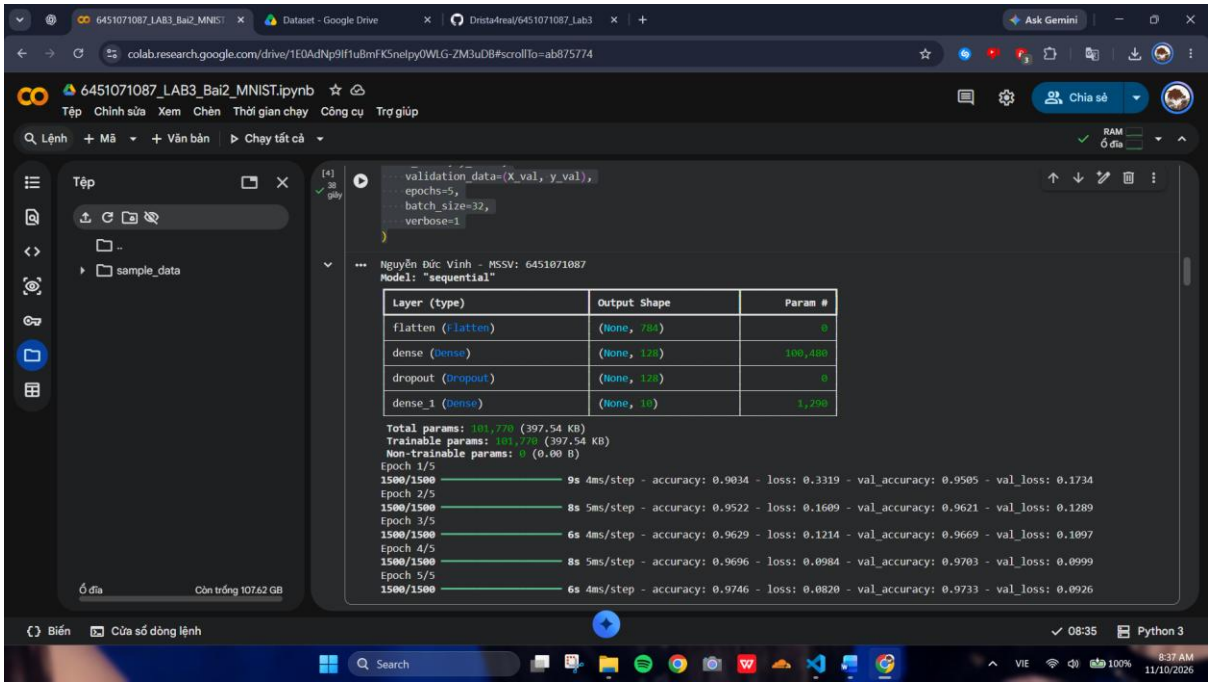
X_train, X_val, y_train, y_val = train_test_split(
    train_images, train_labels,
```

```
    test_size=0.2, random_state=42, stratify=train_labels
)
print("X_train:", X_train.shape, "y_train:", y_train.shape)
print("X_val:", X_val.shape, "y_val:", y_val.shape)
```



Xây dựng và huấn luyện Neural Network (câu e, f, g):

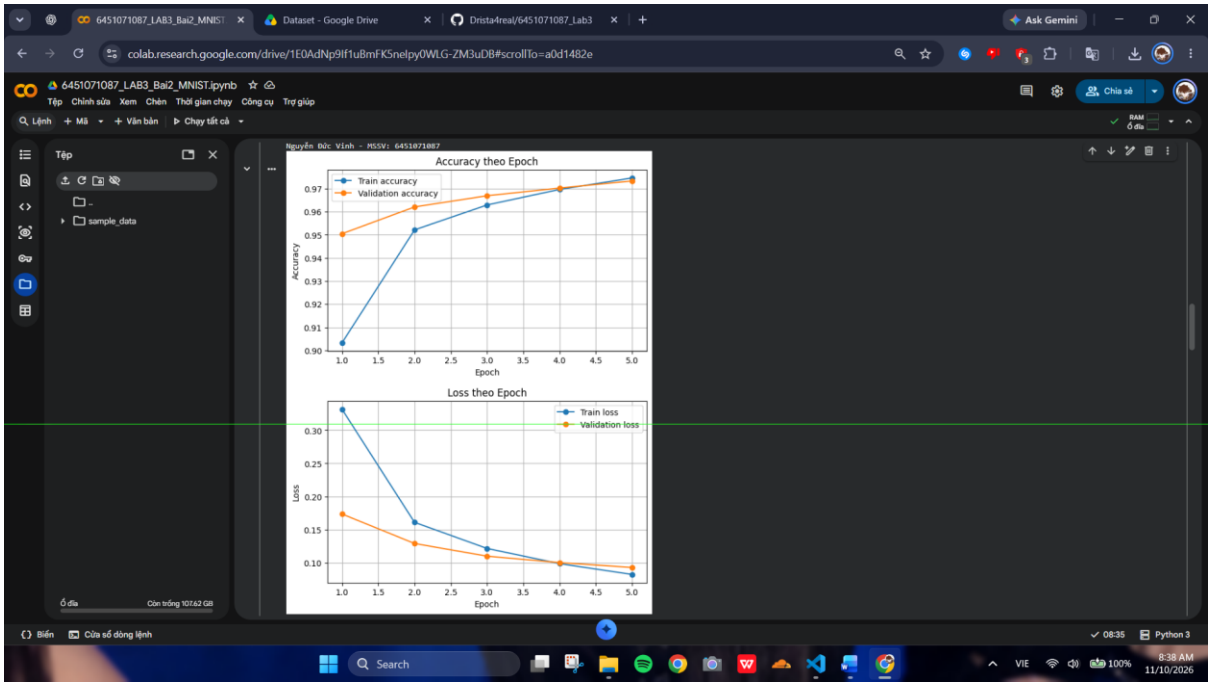
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.summary()
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=5,
    batch_size=32,
    verbose=1
)
```



Biểu đồ Accuracy và Loss (câu h):

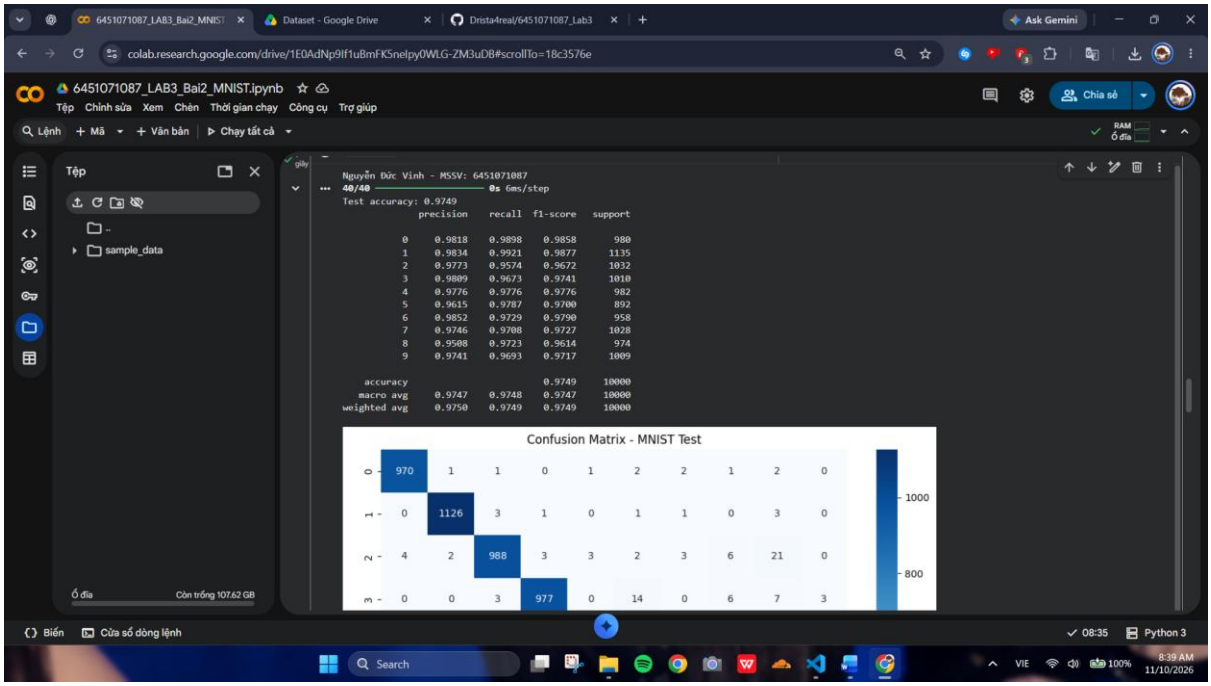
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
epochs = range(1, len(history.history["accuracy"]) + 1)
plt.figure(figsize=(7, 4))
plt.plot(epochs, history.history["accuracy"], marker="o", label="Train accuracy")
plt.plot(epochs, history.history["val_accuracy"], marker="o", label="Validation
accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy theo Epoch")
plt.legend()
plt.grid(True)
plt.show()

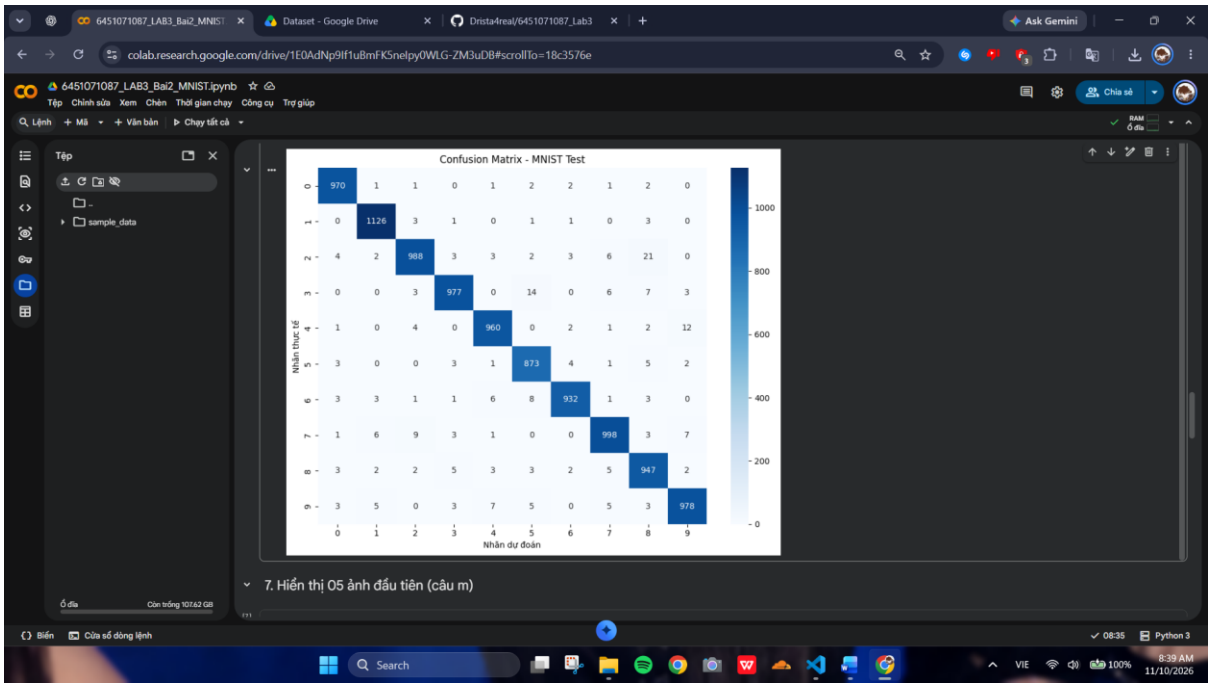
plt.figure(figsize=(7, 4))
plt.plot(epochs, history.history["loss"], marker="o", label="Train loss")
plt.plot(epochs, history.history["val_loss"], marker="o", label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss theo Epoch")
plt.legend()
plt.grid(True)
plt.show()
```



Phân lớp tập test và đánh giá (câu i, j, k):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
test_prob = model.predict(test_images, batch_size=256, verbose=1)
test_pred = np.argmax(test_prob, axis=1)
print("Test accuracy:", round(accuracy_score(test_labels, test_pred), 4))
print(classification_report(test_labels, test_pred,
                            labels=np.arange(10), digits=4, zero_division=0))
cm = confusion_matrix(test_labels, test_pred, labels=np.arange(10))
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=np.arange(10), yticklabels=np.arange(10))
plt.xlabel("Nhãn dự đoán")
plt.ylabel("Nhãn thực tế")
plt.title("Confusion Matrix - MNIST Test")
plt.tight_layout()
plt.show()
```

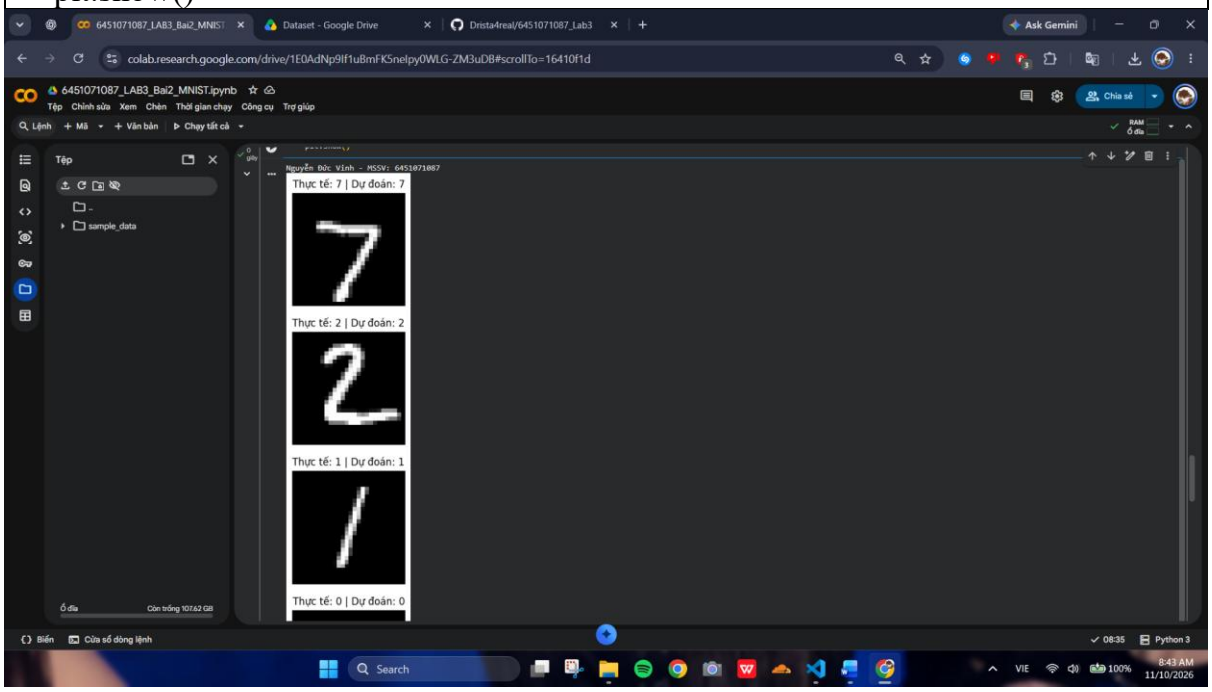


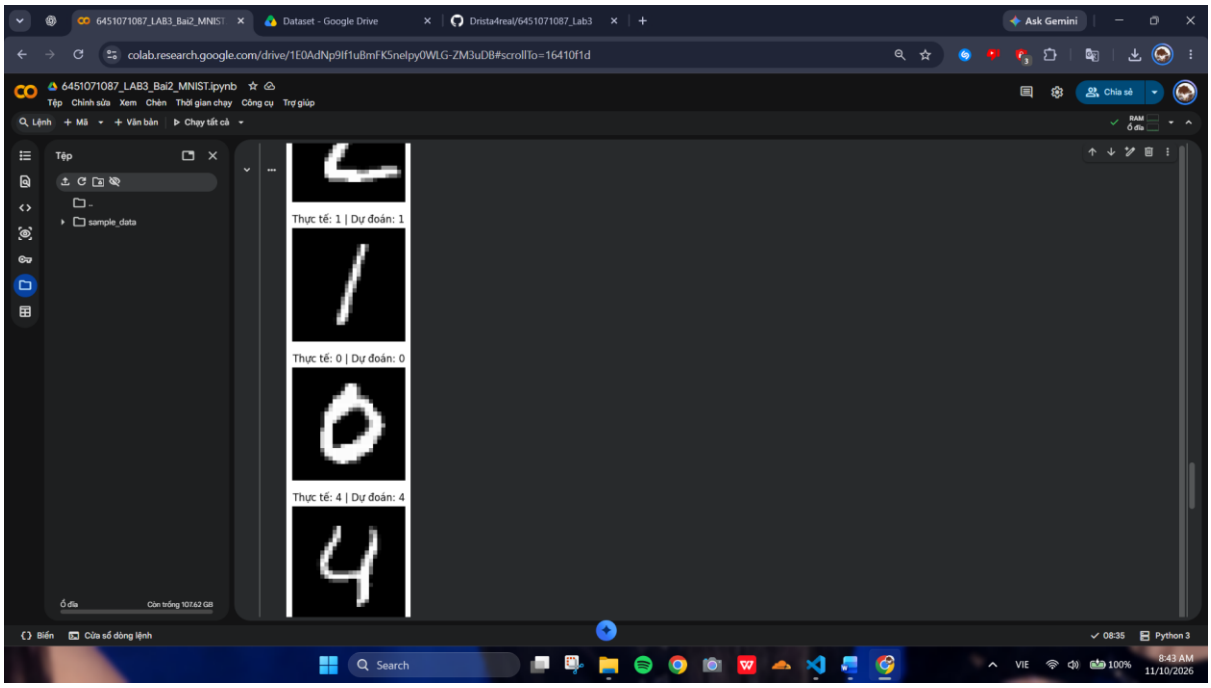


Hiển thị 05 ảnh đầu tiên (câu m):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
for index in range(5):
    image = np.expand_dims(test_images[index], axis=0)
    probabilities = model.predict(image, verbose=0)
    predicted = int(np.argmax(probabilities, axis=1)[0])
    actual = int(test_labels[index])
    plt.figure(figsize=(2.5, 2.5))
    plt.imshow(test_images[index], cmap="gray")
    plt.title(f"Thực tế: {actual} | Dự đoán: {predicted}")
```

```
    plt.axis("off")
    plt.show()
```

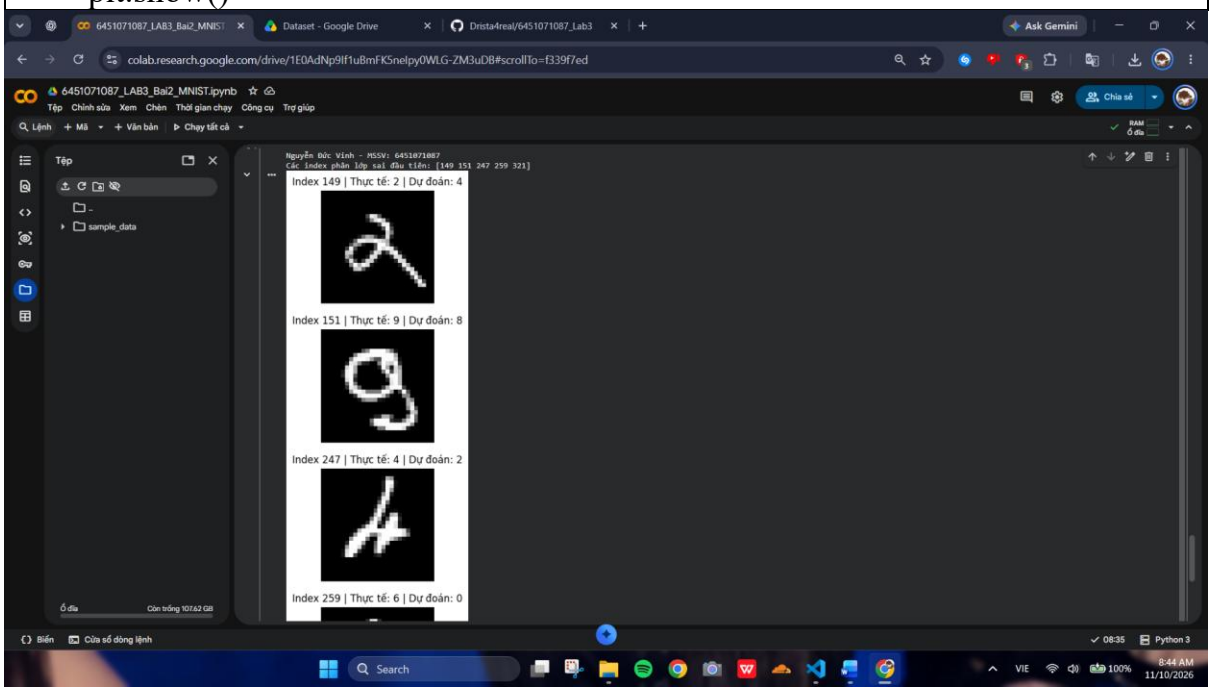


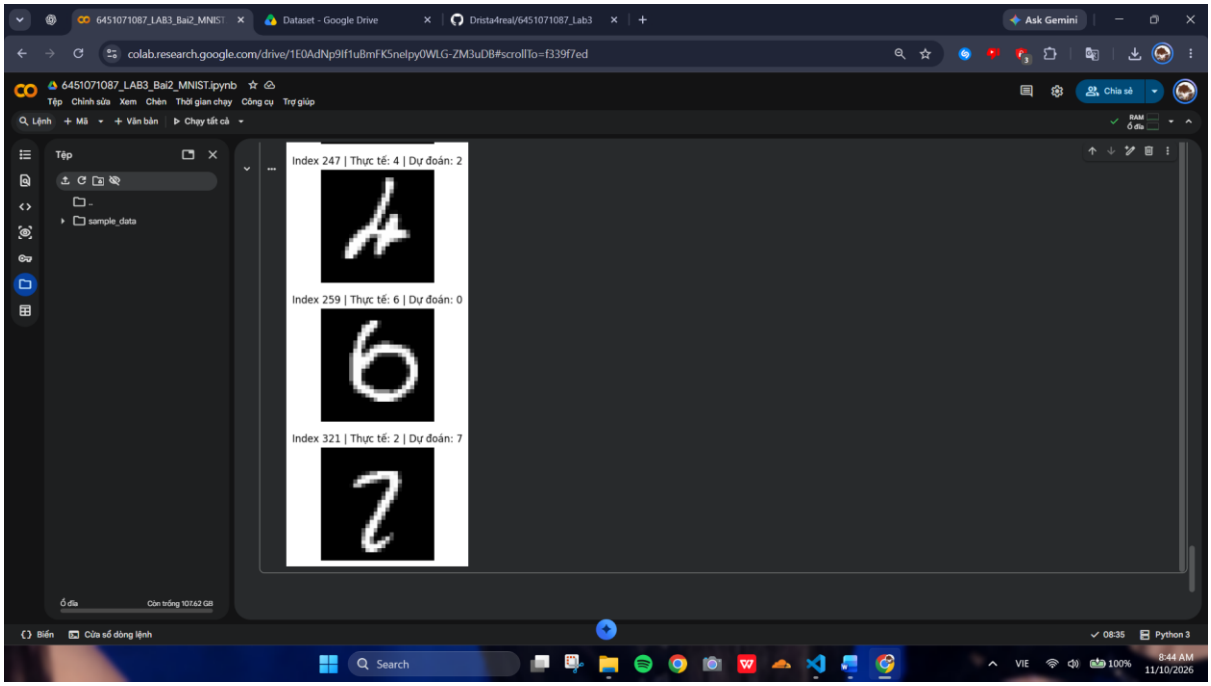


Hiển thị 05 trường hợp phân lớp sai đầu tiên (câu n):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
wrong_indices = np.where(test_pred != test_labels)[0][:5]
print("Các index phân lớp sai đầu tiên:", wrong_indices)
if len(wrong_indices) == 0:
    print("Không có mẫu phân lớp sai trong tập test.")
else:
    for index in wrong_indices:
```

```
        plt.figure(figsize=(2.5, 2.5))
        plt.imshow(test_images[index], cmap="gray")
        plt.title(f"Index {index} | Thực tế: {test_labels[index]} | Dự đoán:
{test_pred[index]}")
        plt.axis("off")
        plt.show()
```





Bài 3 (Trang 7) – MNIST 10-Fold Cross Validation

Dữ liệu MNIST được chuẩn hóa về [0,1]. Hàm create_model xây dựng mô hình tương tự Bài 2. KFold dùng n_splits=10, shuffle=True, random_state=42.

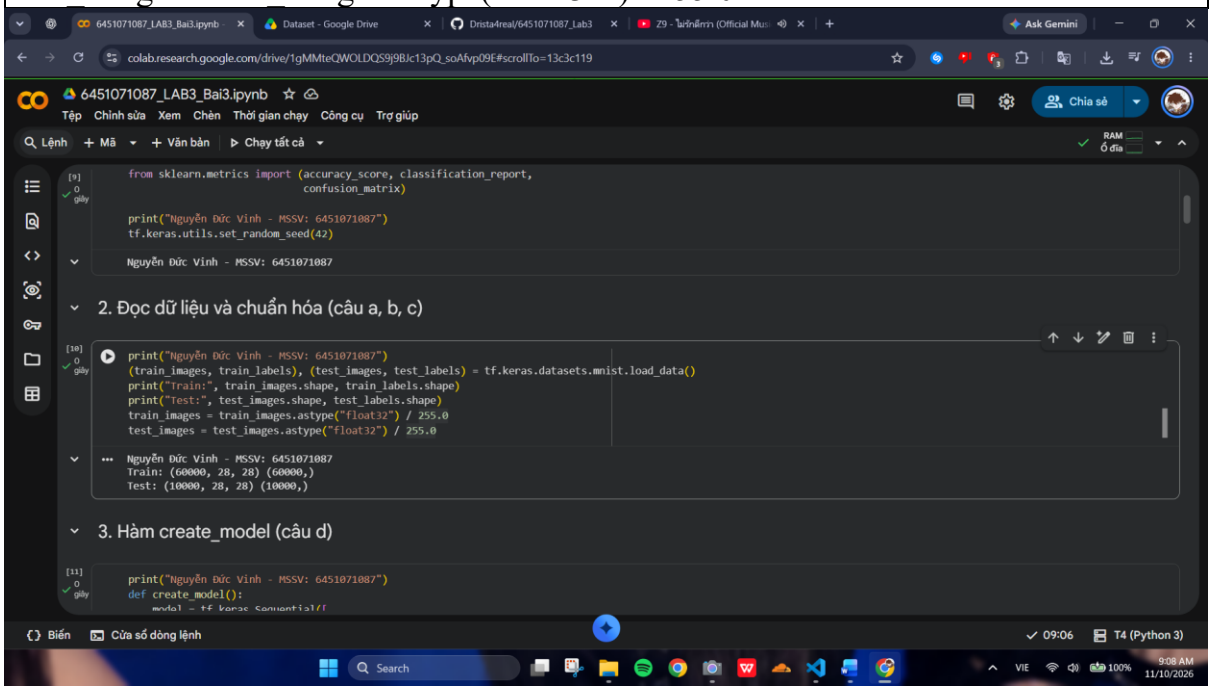
Mỗi fold huấn luyện 5 epoch với batch_size=32, đánh giá trên fold validation và lưu mô hình riêng. Mỗi fold lưu validation loss và accuracy.

Sau 10 fold, tính Mean/Standard Deviation của Accuracy và Loss; vẽ biểu đồ theo fold; chọn mô hình có accuracy validation cao nhất để đánh giá tập test bằng Accuracy, Precision, Recall, F1-score và confusion matrix.

Mã nguồn: Notebook: 6451071087_LAB3_Bai3.ipynb

Đọc dữ liệu và chuẩn hóa (câu a, b, c):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()
print("Train:", train_images.shape, train_labels.shape)
print("Test:", test_images.shape, test_labels.shape)
train_images = train_images.astype("float32") / 255.0
test_images = test_images.astype("float32") / 255.0
```



Hàm create_model (câu d):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(10, activation="softmax")
    ])
    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
```

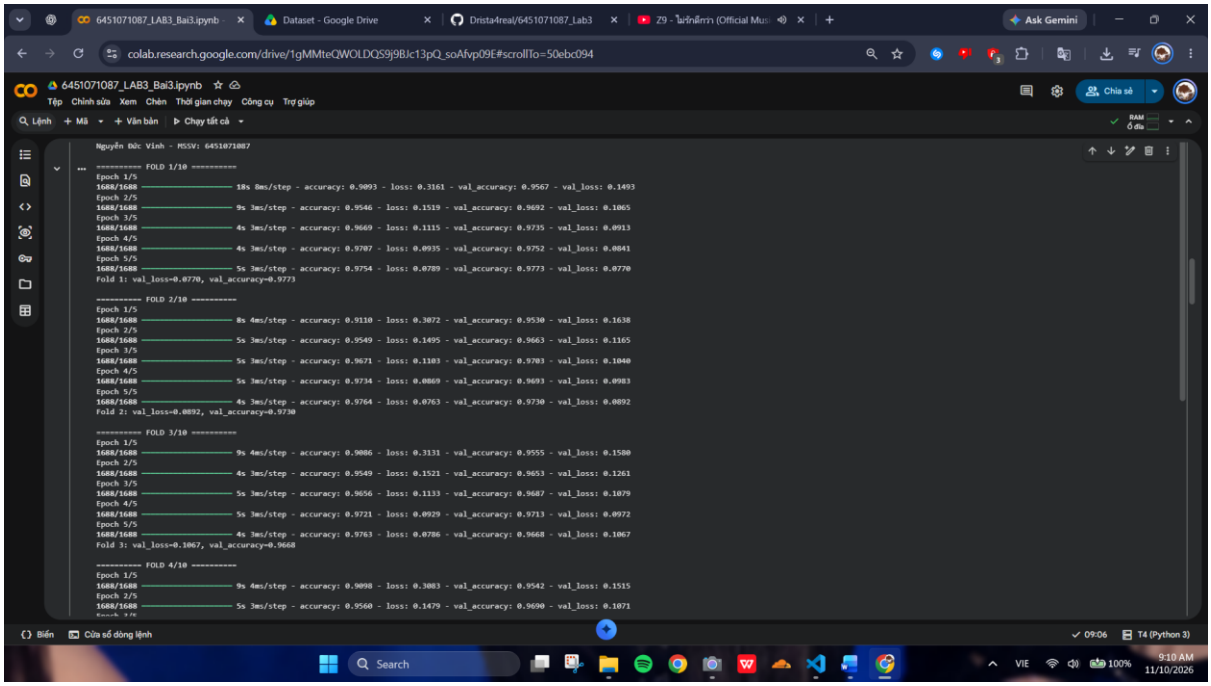
```
            metrics=["accuracy"])
    return model
```

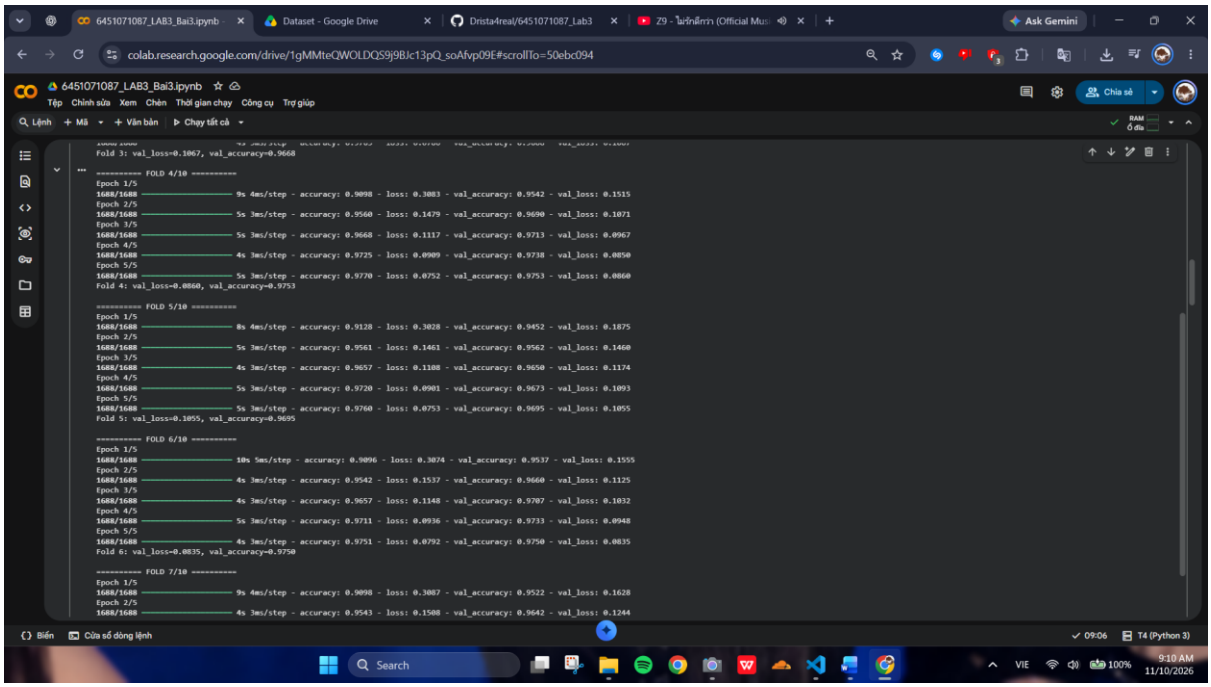
Huấn luyện 10-fold cross validation và lưu mô hình (câu e):

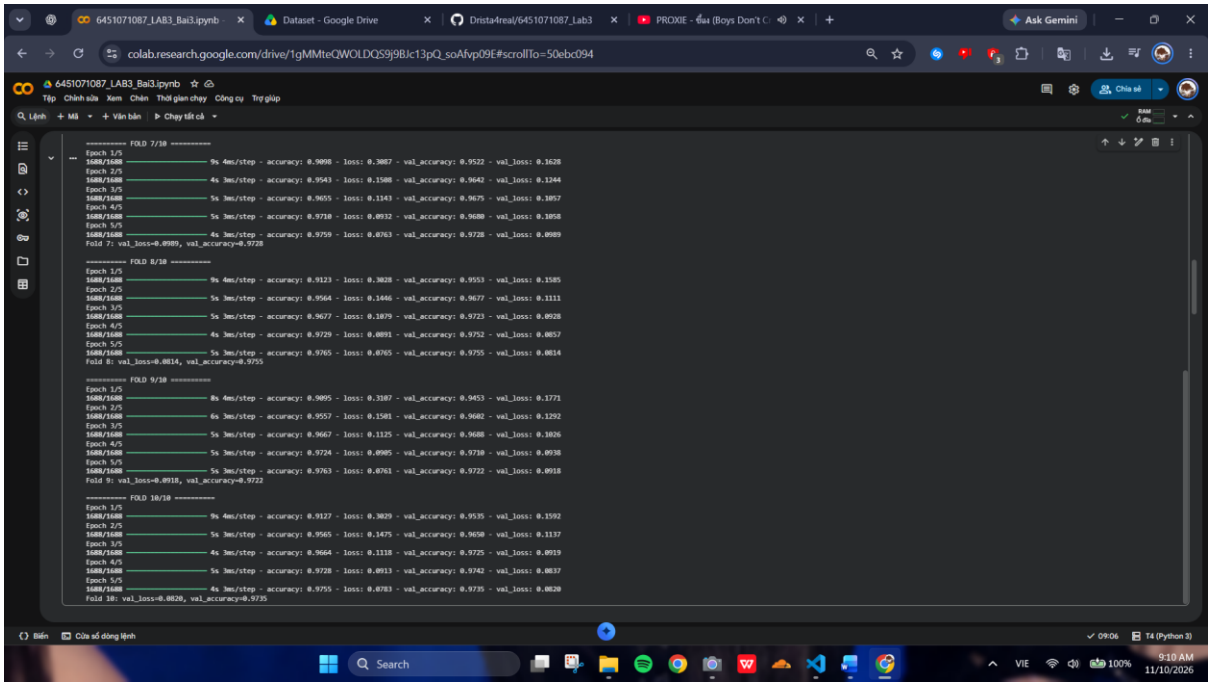
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
kfold = KFold(n_splits=10, shuffle=True, random_state=42)
accuracy_per_fold = []
loss_per_fold = []
models = []
fold_histories = []
model_dir = "/content/6451071087_LAB3_Bai3_models"
os.makedirs(model_dir, exist_ok=True)

for fold, (train_index, val_index) in enumerate(kfold.split(train_images), start=1):
    print(f"\n========== FOLD {fold}/10 ==========")
    X_train_fold = train_images[train_index]
    X_val_fold = train_images[val_index]
    y_train_fold = train_labels[train_index]
    y_val_fold = train_labels[val_index]

    tf.keras.backend.clear_session()
    model = create_model()
    history = model.fit(
        X_train_fold, y_train_fold,
        validation_data=(X_val_fold, y_val_fold),
        epochs=5,
        batch_size=32,
        verbose=1
    )
    val_loss, val_accuracy = model.evaluate(X_val_fold, y_val_fold, verbose=0)
    model.save(os.path.join(model_dir, f"fold_{fold}.keras"))
    models.append(model)
    fold_histories.append(history.history)
    loss_per_fold.append(float(val_loss))
    accuracy_per_fold.append(float(val_accuracy))
    print(f"Fold {fold}: val_loss={val_loss:.4f}, val_accuracy={val_accuracy:.4f}")
```

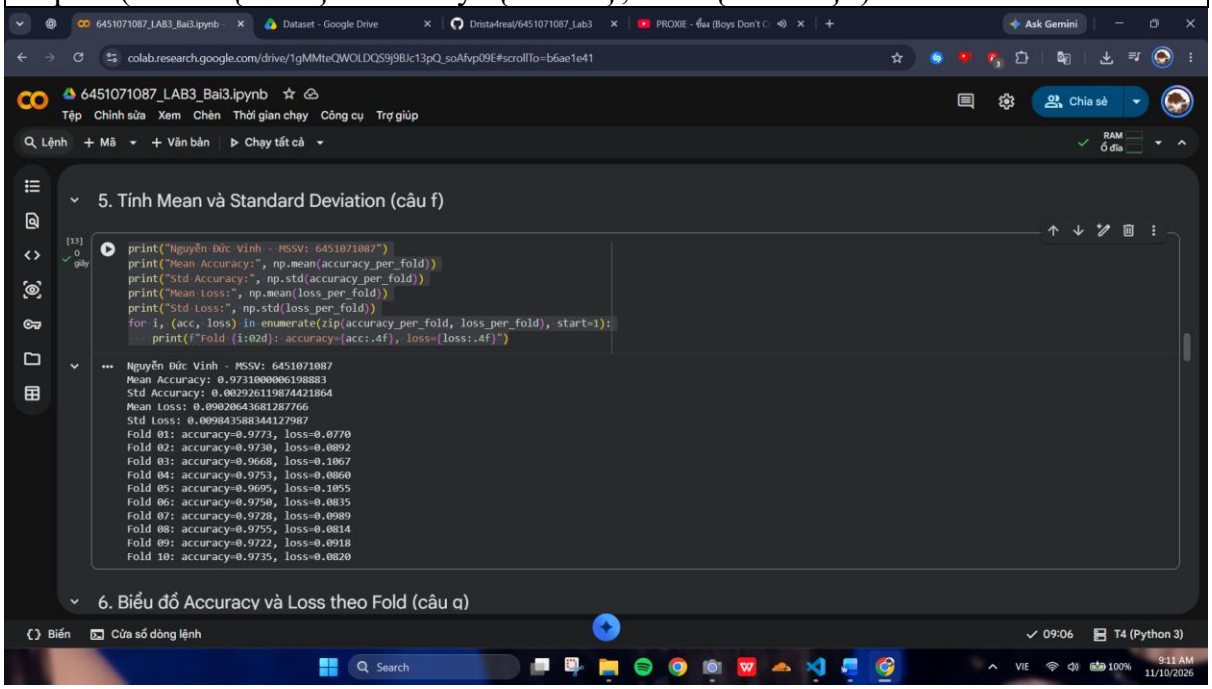

6451071087_LAB3_Bai3.ipynb
Dataset - Google Drive
colab.research.google.com/drive/1gMMteQWOLDQS9j9BJc13pQ_soAfvp09E#scrollTo=50ebc094
Nguyễn Đức Vinh - MSSV: 6451071087
========== FOLD 1/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 18s 8ms/step - accuracy: 0.9093 - loss: 0.3161 - val_accuracy: 0.9567 - val_loss: 0.1493
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step - accuracy: 0.9546 - loss: 0.1519 - val_accuracy: 0.9692 - val_loss: 0.1065
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9669 - loss: 0.1115 - val_accuracy: 0.9735 - val_loss: 0.0913
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9707 - loss: 0.0935 - val_accuracy: 0.9752 - val_loss: 0.0841
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9754 - loss: 0.0789 - val_accuracy: 0.9773 - val_loss: 0.0770
Fold 1: val_loss=0.0770, val_accuracy=0.9773
========== FOLD 2/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9110 - loss: 0.3072 - val_accuracy: 0.9530 - val_loss: 0.1638
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9549 - loss: 0.1495 - val_accuracy: 0.9663 - val_loss: 0.1165
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9671 - loss: 0.1103 - val_accuracy: 0.9703 - val_loss: 0.1040
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9734 - loss: 0.0869 - val_accuracy: 0.9693 - val_loss: 0.0983
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9764 - loss: 0.0763 - val_accuracy: 0.9730 - val_loss: 0.0892
Fold 2: val_loss=0.0892, val_accuracy=0.9730
========== FOLD 3/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9086 - loss: 0.3131 - val_accuracy: 0.9555 - val_loss: 0.1580
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9549 - loss: 0.1521 - val_accuracy: 0.9653 - val_loss: 0.1261
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9656 - loss: 0.1133 - val_accuracy: 0.9687 - val_loss: 0.1079
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9721 - loss: 0.0929 - val_accuracy: 0.9713 - val_loss: 0.0972
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9763 - loss: 0.0786 - val_accuracy: 0.9668 - val_loss: 0.1067
Fold 3: val_loss=0.1067, val_accuracy=0.9668
========== FOLD 4/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9098 - loss: 0.3083 - val_accuracy: 0.9542 - val_loss: 0.1515
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9560 - loss: 0.1479 - val_accuracy: 0.9690 - val_loss: 0.1071
Biến
Cửa sổ dòng lệnh
09:06
T4 (Python 3)
Search
9:10 AM
11/10/2026


6451071087_LAB3_Bai3.ipynb
Dataset - Google Drive
colab.research.google.com/drive/1gMMteQWOLDQS9j9BJc13pQ_soAfvp09E#scrollTo=50ebc094
Fold 3: val_loss=0.1067, val_accuracy=0.9668
========== FOLD 4/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9098 - loss: 0.3083 - val_accuracy: 0.9542 - val_loss: 0.1515
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9560 - loss: 0.1479 - val_accuracy: 0.9690 - val_loss: 0.1071
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9668 - loss: 0.1117 - val_accuracy: 0.9713 - val_loss: 0.0967
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9725 - loss: 0.0909 - val_accuracy: 0.9738 - val_loss: 0.0850
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9770 - loss: 0.0752 - val_accuracy: 0.9753 - val_loss: 0.0860
Fold 4: val_loss=0.0860, val_accuracy=0.9753
========== FOLD 5/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9128 - loss: 0.3028 - val_accuracy: 0.9452 - val_loss: 0.1875
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9561 - loss: 0.1461 - val_accuracy: 0.9562 - val_loss: 0.1460
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9657 - loss: 0.1108 - val_accuracy: 0.9650 - val_loss: 0.1174
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9720 - loss: 0.0901 - val_accuracy: 0.9673 - val_loss: 0.1093
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9760 - loss: 0.0753 - val_accuracy: 0.9695 - val_loss: 0.1055
Fold 5: val_loss=0.1055, val_accuracy=0.9695
========== FOLD 6/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9096 - loss: 0.3074 - val_accuracy: 0.9537 - val_loss: 0.1555
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9542 - loss: 0.1537 - val_accuracy: 0.9660 - val_loss: 0.1125
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9657 - loss: 0.1148 - val_accuracy: 0.9707 - val_loss: 0.1032
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9711 - loss: 0.0936 - val_accuracy: 0.9733 - val_loss: 0.0948
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9751 - loss: 0.0792 - val_accuracy: 0.9750 - val_loss: 0.0835
Fold 6: val_loss=0.0835, val_accuracy=0.9750
========== FOLD 7/10 ==========
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9098 - loss: 0.3087 - val_accuracy: 0.9522 - val_loss: 0.1628
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9543 - loss: 0.1508 - val_accuracy: 0.9642 - val_loss: 0.1244
Biến
Cửa sổ dòng lệnh
09:06
T4 (Python 3)
Search
9:10 AM
11/10/2026



Tính Mean và Standard Deviation (câu f):

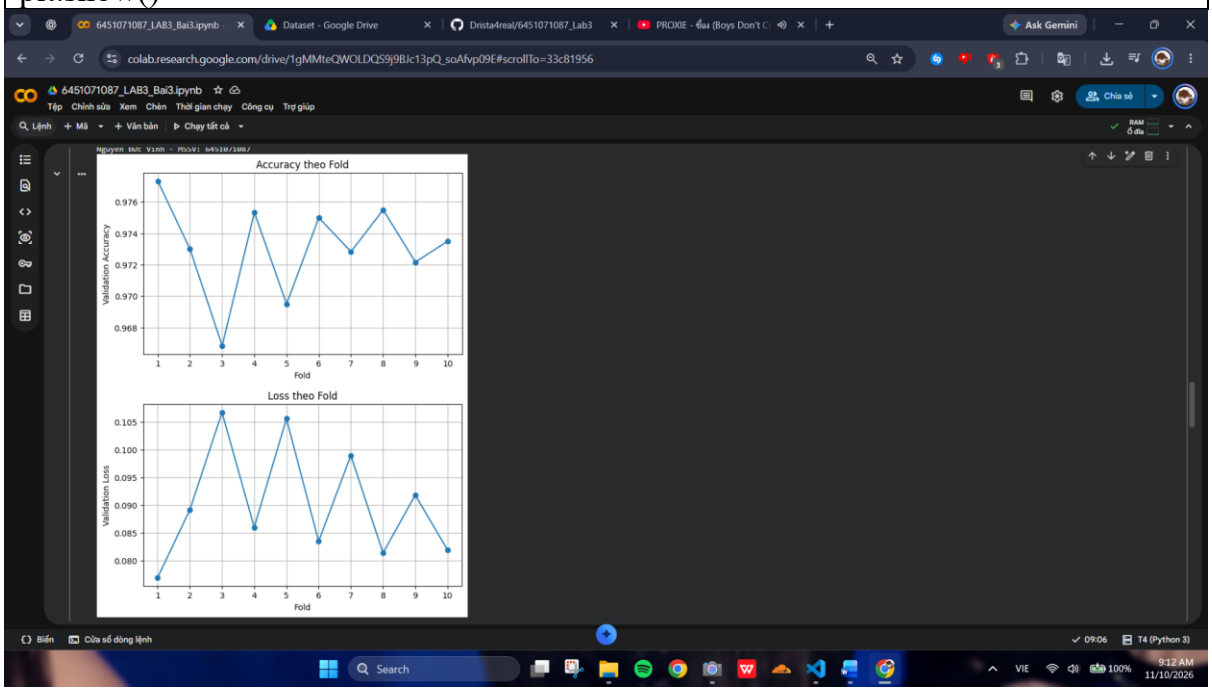
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
print("Mean Accuracy:", np.mean(accuracy_per_fold))
print("Std Accuracy:", np.std(accuracy_per_fold))
print("Mean Loss:", np.mean(loss_per_fold))
print("Std Loss:", np.std(loss_per_fold))
for i, (acc, loss) in enumerate(zip(accuracy_per_fold, loss_per_fold), start=1):
    print(f"Fold {i:02d}: accuracy={acc:.4f}, loss={loss:.4f}")
```



Biểu đồ Accuracy và Loss theo Fold (câu g):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
fold_numbers = np.arange(1, 11)
plt.figure(figsize=(7, 4))
plt.plot(fold_numbers, accuracy_per_fold, marker="o")
plt.xlabel("Fold")
plt.ylabel("Validation Accuracy")
plt.title("Accuracy theo Fold")
plt.xticks(fold_numbers)
plt.grid(True)
plt.show()

plt.figure(figsize=(7, 4))
plt.plot(fold_numbers, loss_per_fold, marker="o")
plt.xlabel("Fold")
plt.ylabel("Validation Loss")
plt.title("Loss theo Fold")
plt.xticks(fold_numbers)
plt.grid(True)
plt.show()
```

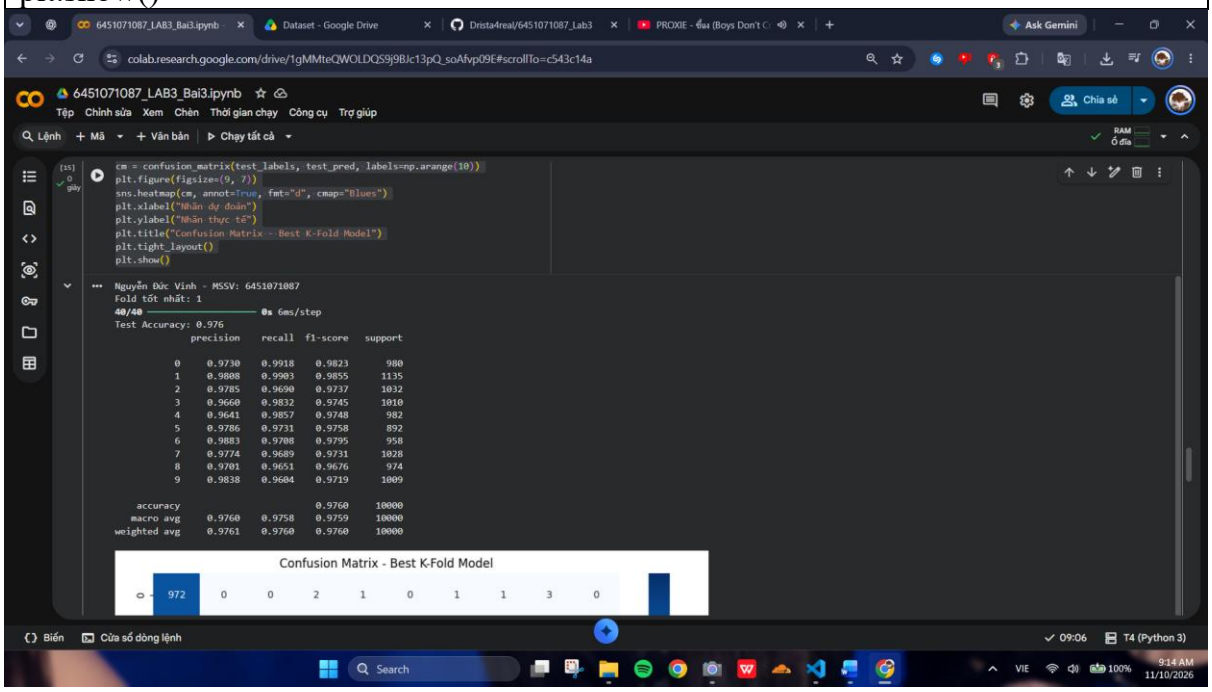


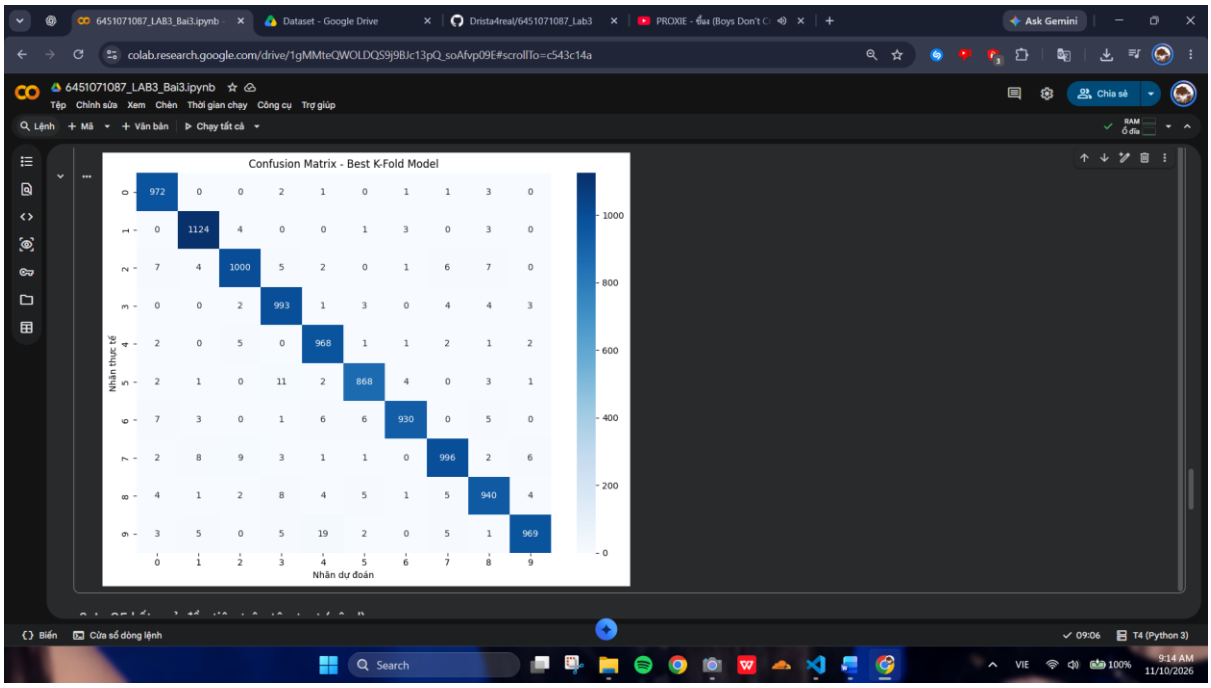
Chọn mô hình có Accuracy cao nhất và đánh giá tập test (câu h, i, j, k):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
best_index = int(np.argmax(accuracy_per_fold))
best_model = models[best_index]
print("Fold tốt nhất:", best_index + 1)

test_prob = best_model.predict(test_images, batch_size=256, verbose=1)
test_pred = np.argmax(test_prob, axis=1)
print("Test Accuracy:", accuracy_score(test_labels, test_pred))
```

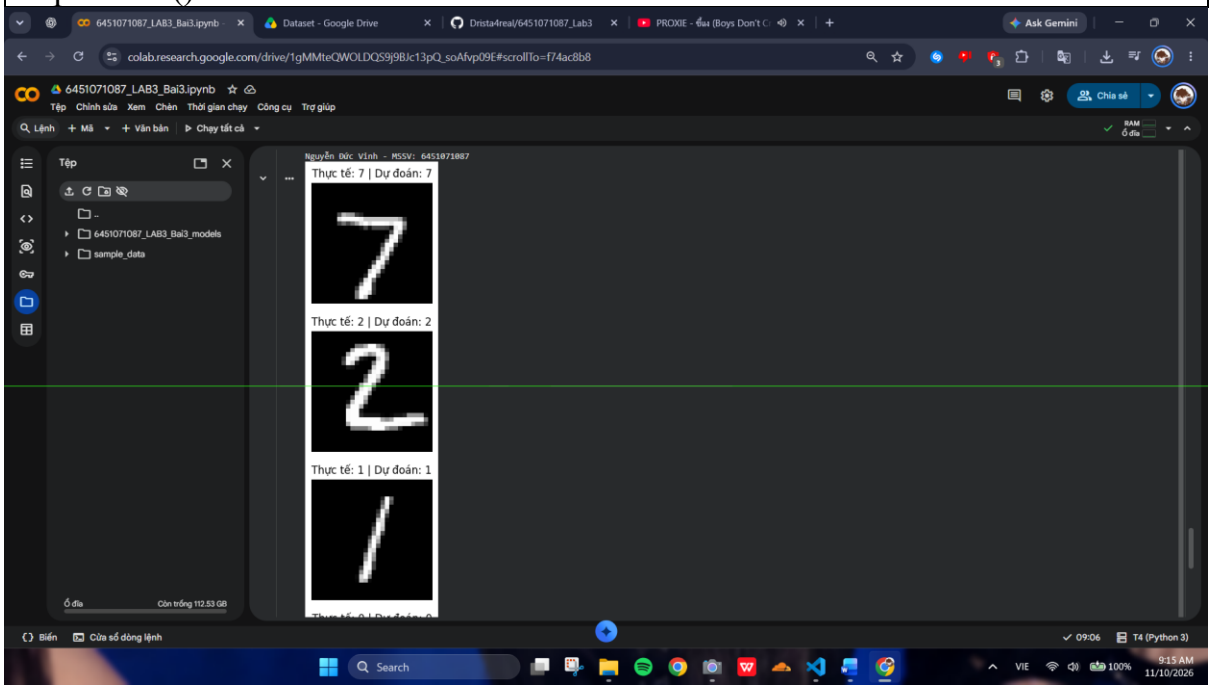
```
print(classification_report(test_labels, test_pred,
                            labels=np.arange(10), digits=4, zero_division=0))
cm = confusion_matrix(test_labels, test_pred, labels=np.arange(10))
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Nhãn dự đoán")
plt.ylabel("Nhãn thực tế")
plt.title("Confusion Matrix - Best K-Fold Model")
plt.tight_layout()
plt.show()
```

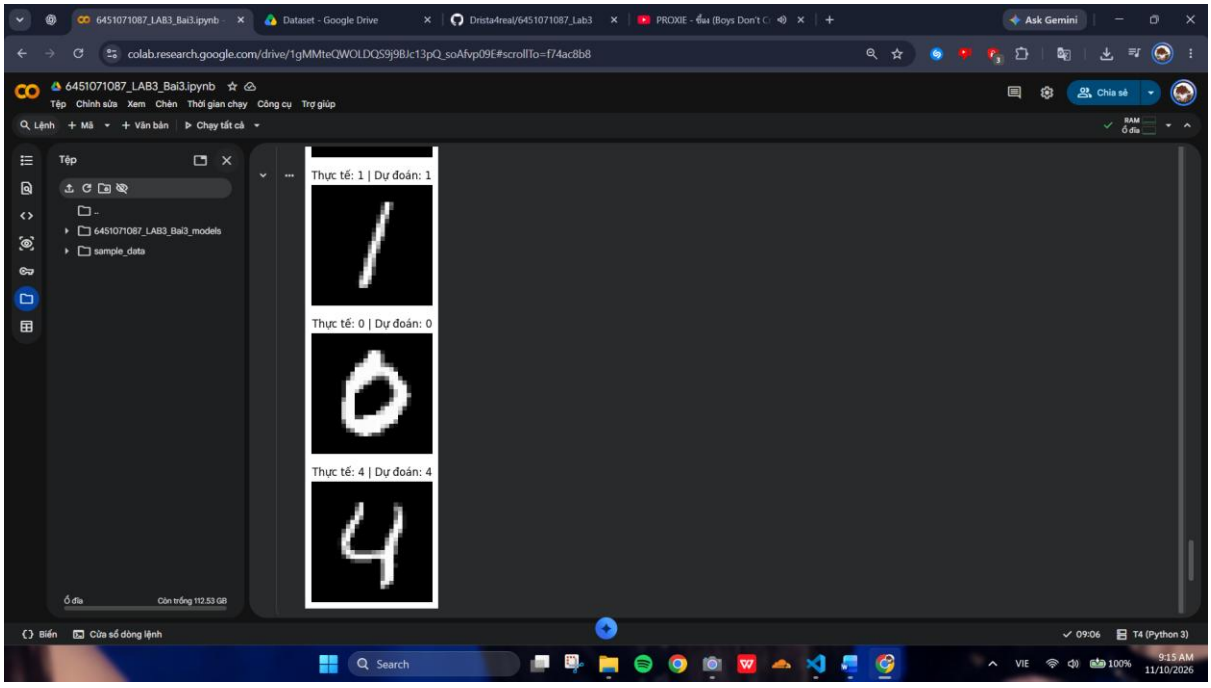




In 05 kết quả đầu tiên trên tập test (câu l):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
for index in range(5):
    predicted = int(test_pred[index])
    actual = int(test_labels[index])
    plt.figure(figsize=(2.5, 2.5))
    plt.imshow(test_images[index], cmap="gray")
    plt.title(f"Thực tế: {actual} | Dự đoán: {predicted}")
    plt.axis("off")
    plt.show()
```





Bài 4 (Trang 9) – Iris 10-fold Cross Validation

Đọc Iris từ sklearn và chuẩn hóa theo max từng thuộc tính. Hàm create_model gồm 10 neuron ReLU, 20 neuron ReLU và 3 neuron Softmax; compile với Adam, sparse_categorical_crossentropy và accuracy.

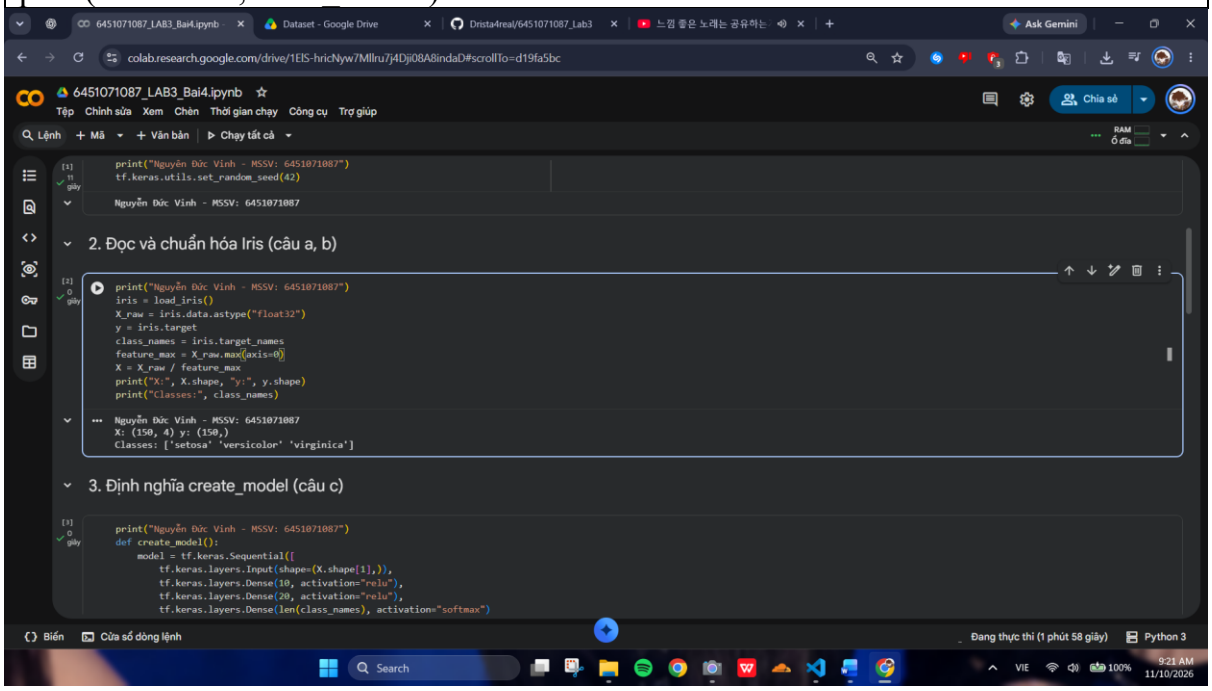
KFold sử dụng 10 fold, shuffle=True, random_state=42. Mỗi fold huấn luyện 100 epoch với batch_size=1; lưu mô hình và tính Accuracy, Precision, Recall, F1-score.

Tính trung bình các độ đo trên 10 fold và trung bình Precision/Recall/F1-score theo từng lớp. Chọn mô hình có macro F1-score cao nhất và dự đoán ba mẫu Iris mới.

Mã nguồn: Notebook: 6451071087_LAB3_Bai4.ipynb

Đọc và chuẩn hóa Iris (câu a, b):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
iris = load_iris()
X_raw = iris.data.astype("float32")
y = iris.target
class_names = iris.target_names
feature_max = X_raw.max(axis=0)
X = X_raw / feature_max
print("X:", X.shape, "y:", y.shape)
print("Classes:", class_names)
```



Định nghĩa create_model (câu c):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X.shape[1],)),
        tf.keras.layers.Dense(10, activation="relu"),
```

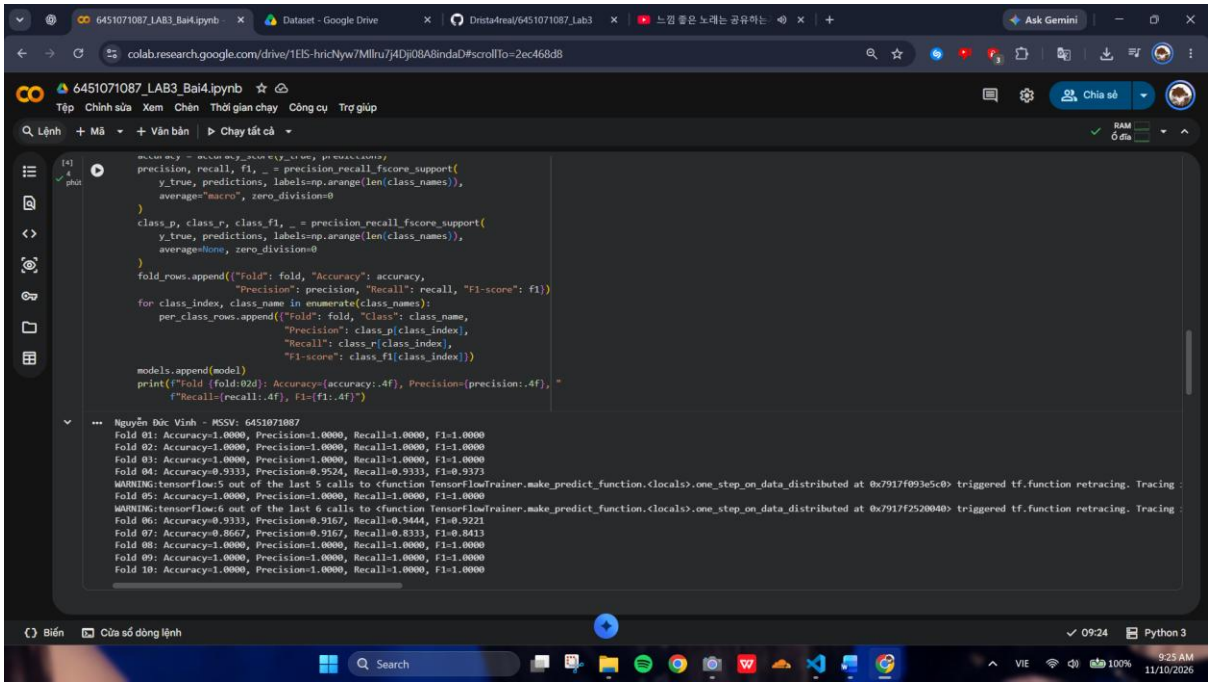
```
        tf.keras.layers.Dense(20, activation="relu"),
        tf.keras.layers.Dense(len(class_names), activation="softmax")
    ])
    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    return model
```

Huấn luyện 10-fold và lưu chỉ số từng fold (câu d):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
kfold = KFold(n_splits=10, shuffle=True, random_state=42)
models = []
fold_rows = []
per_class_rows = []

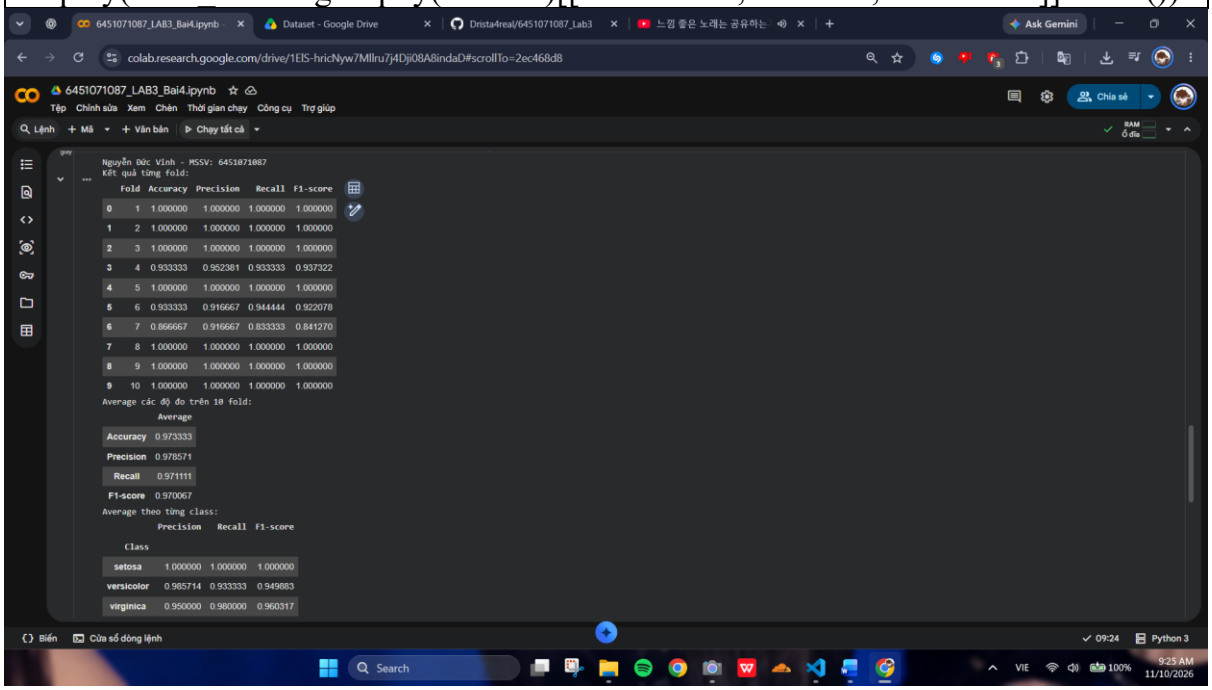
for fold, (train_index, val_index) in enumerate(kfold.split(X), start=1):
    tf.keras.backend.clear_session()
    model = create_model()
    model.fit(X[train_index], y[train_index], epochs=100,
              batch_size=1, verbose=0)
    probabilities = model.predict(X[val_index], verbose=0)
    predictions = np.argmax(probabilities, axis=1)
    y_true = y[val_index]

    accuracy = accuracy_score(y_true, predictions)
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_true, predictions, labels=np.arange(len(class_names)),
        average="macro", zero_division=0
    )
    class_p, class_r, class_f1, _ = precision_recall_fscore_support(
        y_true, predictions, labels=np.arange(len(class_names)),
        average=None, zero_division=0
    )
    fold_rows.append({"Fold": fold, "Accuracy": accuracy,
                      "Precision": precision, "Recall": recall, "F1-score": f1})
    for class_index, class_name in enumerate(class_names):
        per_class_rows.append({"Fold": fold, "Class": class_name,
                               "Precision": class_p[class_index],
                               "Recall": class_r[class_index],
                               "F1-score": class_f1[class_index]})
    models.append(model)
    print(f"Fold {fold:02d}: Accuracy={accuracy:.4f}, Precision={precision:.4f}, "
          f"Recall={recall:.4f}, F1={f1:.4f}")
```



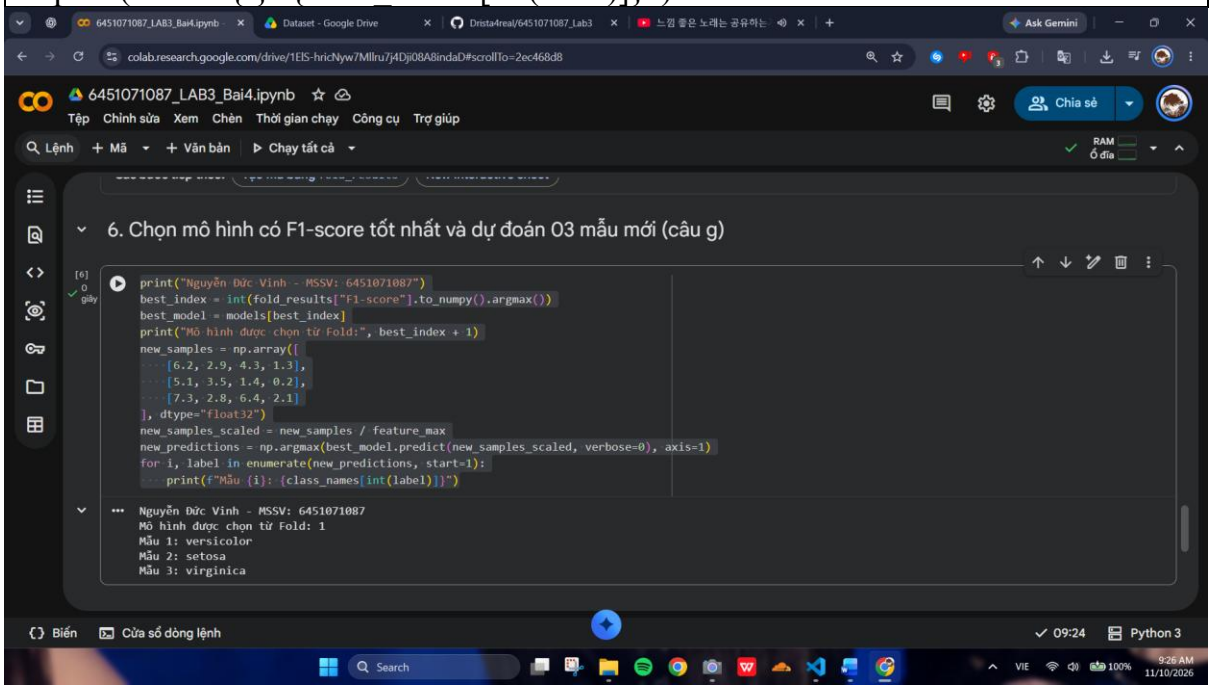
Average các độ đo và độ đo theo từng class (câu e, f):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
fold_results = pd.DataFrame(fold_rows)
class_results = pd.DataFrame(per_class_rows)
print("Kết quả từng fold:")
display(fold_results)
print("Average các độ đo trên 10 fold:")
display(fold_results[["Accuracy", "Precision", "Recall", "F1-
score"]].mean().to_frame("Average"))
print("Average theo từng class:")
display(class_results.groupby("Class")[["Precision", "Recall", "F1-score"]].mean())
```



Chọn mô hình có F1-score tốt nhất và dự đoán 03 mẫu mới (câu g):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
best_index = int(fold_results["F1-score"].to_numpy().argmax())
best_model = models[best_index]
print("Mô hình được chọn từ Fold:", best_index + 1)
new_samples = np.array([
    [6.2, 2.9, 4.3, 1.3],
    [5.1, 3.5, 1.4, 0.2],
    [7.3, 2.8, 6.4, 2.1]
], dtype="float32")
new_samples_scaled = new_samples / feature_max
new_predictions = np.argmax(best_model.predict(new_samples_scaled,
verbose=0), axis=1)
for i, label in enumerate(new_predictions, start=1):
    print(f"Mẫu {i}: {class_names[int(label)]}")
```



Bài 2 (Trang 18 - 19) – Mall Customer Segmentation

Đọc CSV Mall Customer từ thư mục Google Drive; thống kê dữ liệu, kiểm tra và xử lý giá trị thiếu bằng median cho cột số và mode cho giới tính. Vẽ histogram cho Age, Annual Income và Spending Score; thống kê số mẫu theo Gender.

Dùng scatter plot khảo sát Age-Annual Income và Annual Income-Spending Score theo Gender. Với từng nhóm thuộc tính, chuẩn hóa bằng StandardScaler, chạy Elbow trên k=1..10, chọn k tại điểm khuỷu tay, huấn luyện K-Means và tính Silhouette coefficient.

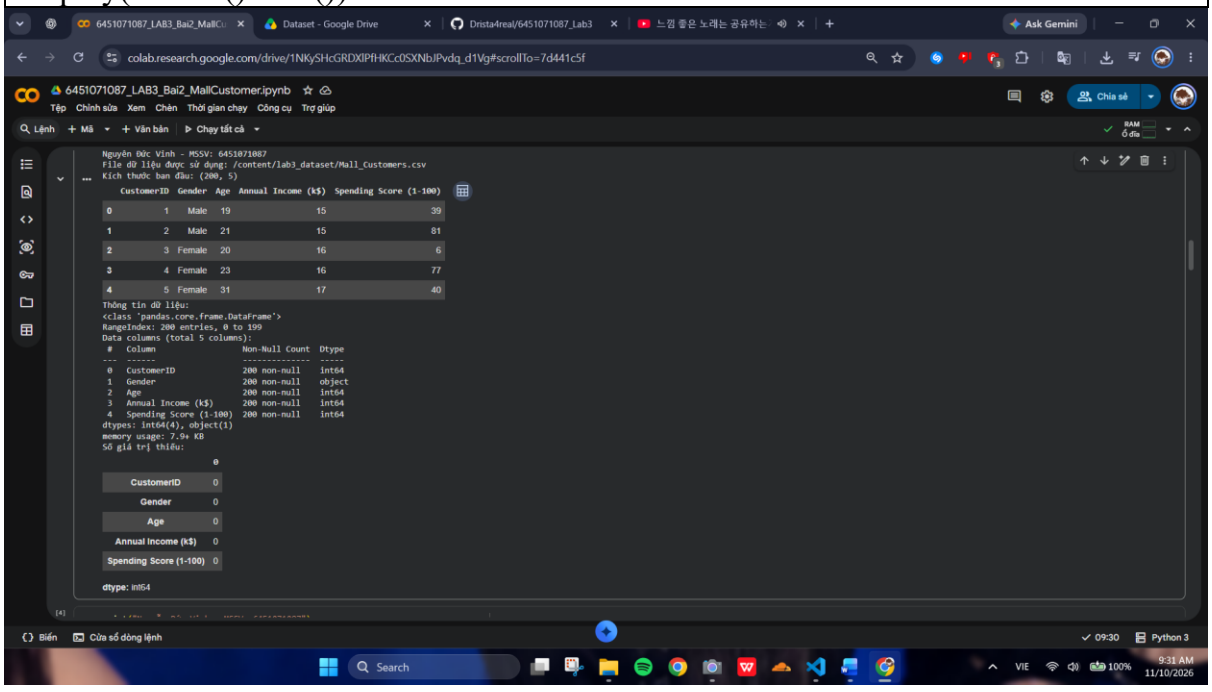
Ba lần phân cụm: (1) Age + Spending Score; (2) Annual Income + Spending Score; (3) Age + Annual Income + Spending Score, trong đó lần thứ ba biểu diễn bằng biểu đồ 3D.

Mã nguồn: Notebook: 6451071087_LAB3_Bai2_MallCustomer.ipynb

Đọc dữ liệu Mall Customer (câu 1):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
candidate_names = {"mall_customers.csv", "mall_customer3.csv",
"mall_customers3.csv"}
csv_files = []
for root in SEARCH_ROOTS:
    if root.exists():
        csv_files.extend([p for p in root.rglob("*.csv")
                          if p.name.lower() in candidate_names or "mall" in p.name.lower()])

if not csv_files:
    raise FileNotFoundError(
        "Không tìm thấy CSV Mall Customer. Kiểm tra quyền chia sẻ thư mục Drive "
        "hoặc đặt file Mall_Customers.csv vào /content/lab3_dataset."
    )
DATA_PATH = csv_files[0]
print("File dữ liệu được sử dụng:", DATA_PATH)
df = pd.read_csv(DATA_PATH)
print("Kích thước ban đầu:", df.shape)
display(df.head())
print("Thông tin dữ liệu:")
df.info()
print("Số giá trị thiếu:")
display(df.isna().sum())
```



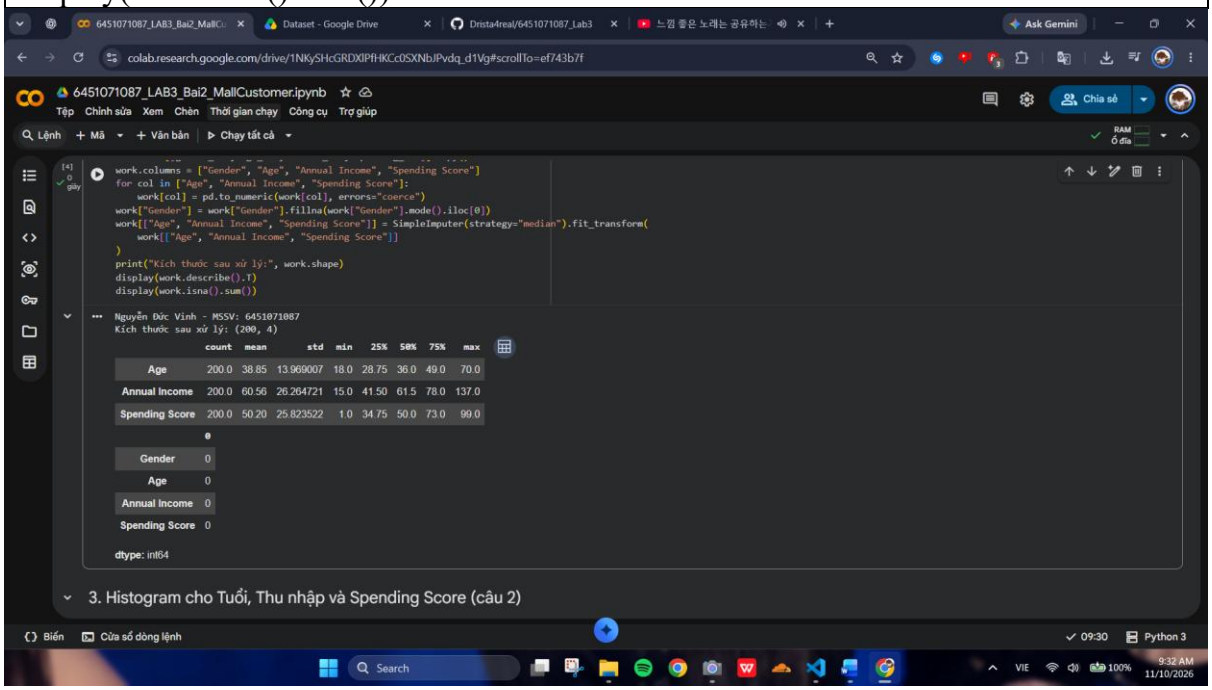

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
# Chuẩn hóa tên cột để tương thích với các cách đặt tên phổ biến trong CSV.
```

```
def norm_col(name):
    return "".join(ch.lower() for ch in str(name) if ch.isalnum())

normalized = {norm_col(c): c for c in df.columns}
def find_column(keyword_options):
    for key, original in normalized.items():
        if any(opt in key for opt in keyword_options):
            return original
    return None

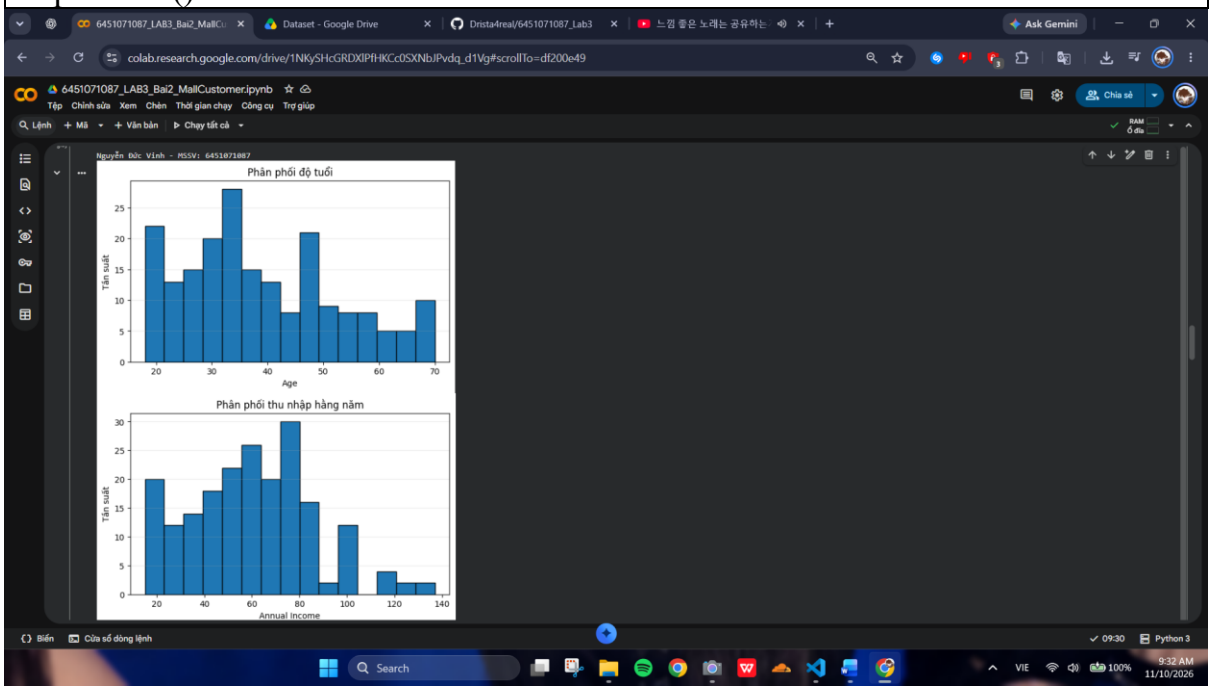
age_col = find_column(["age", "tuoi"])
income_col = find_column(["annualincome", "income"])
spending_col = find_column(["spendingscore", "spending", "diemmuasam"])
gender_col = find_column(["gender", "gioitinh"])
if not all([age_col, income_col, spending_col, gender_col]):
    raise ValueError(f"Không nhận diện đủ cột. Các cột hiện có: {list(df.columns)}")

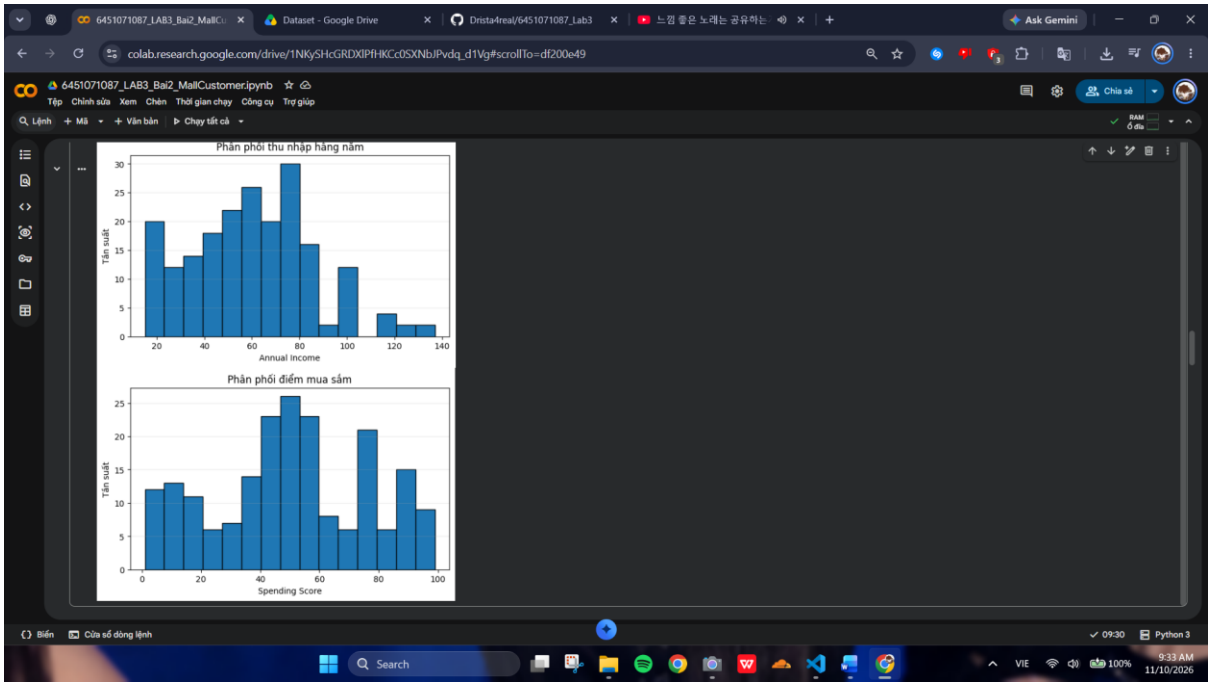
work = df[[gender_col, age_col, income_col, spending_col]].copy()
work.columns = ["Gender", "Age", "Annual Income", "Spending Score"]
for col in ["Age", "Annual Income", "Spending Score"]:
    work[col] = pd.to_numeric(work[col], errors="coerce")
work["Gender"] = work["Gender"].fillna(work["Gender"].mode().iloc[0])
work[["Age", "Annual Income", "Spending Score"]] =
SimpleImputer(strategy="median").fit_transform(
    work[["Age", "Annual Income", "Spending Score"]]
)
print("Kích thước sau xử lý:", work.shape)
display(work.describe().T)
display(work.isna().sum())
```



Histogram cho Tuổi, Thu nhập và Spending Score (câu 2):

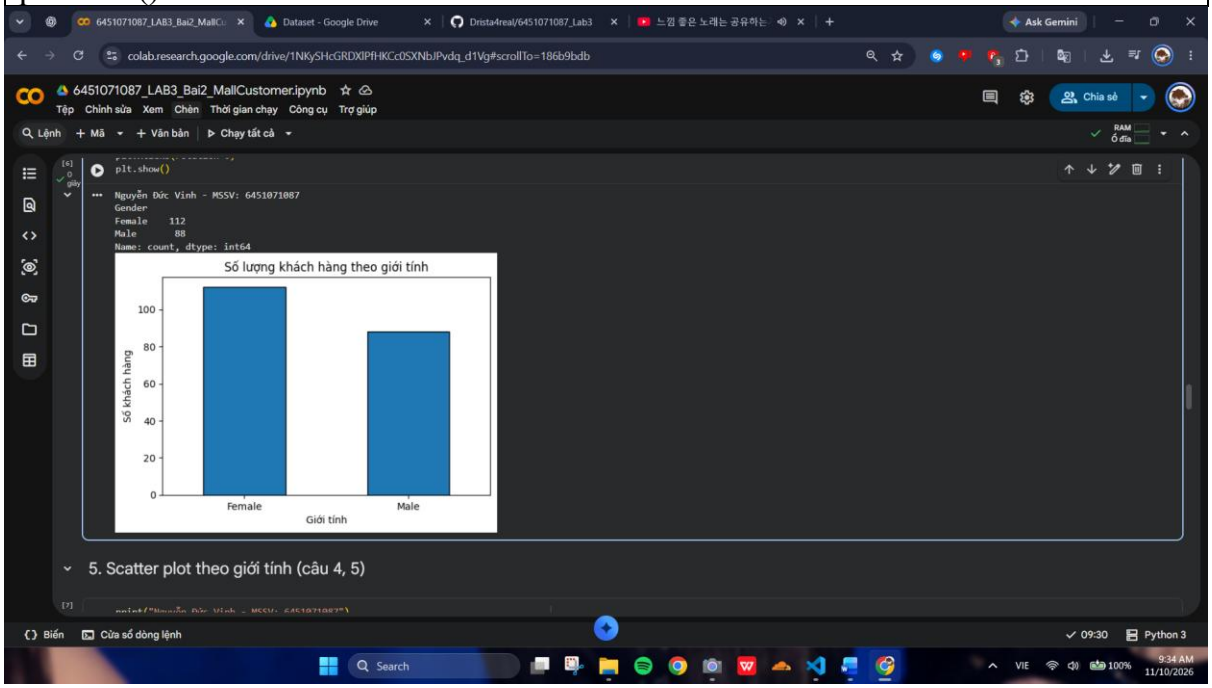
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
for column, title in [("Age", "Phân phối độ tuổi"),
                      ("Annual Income", "Phân phối thu nhập hằng năm"),
                      ("Spending Score", "Phân phối điểm mua sắm")]:
    plt.figure(figsize=(7, 4))
    plt.hist(work[column], bins=15, edgecolor="black")
    plt.xlabel(column)
    plt.ylabel("Tần suất")
    plt.title(title)
    plt.grid(axis="y", alpha=0.25)
    plt.show()
```





Thống kê số lượng mẫu theo giới tính (câu 3):

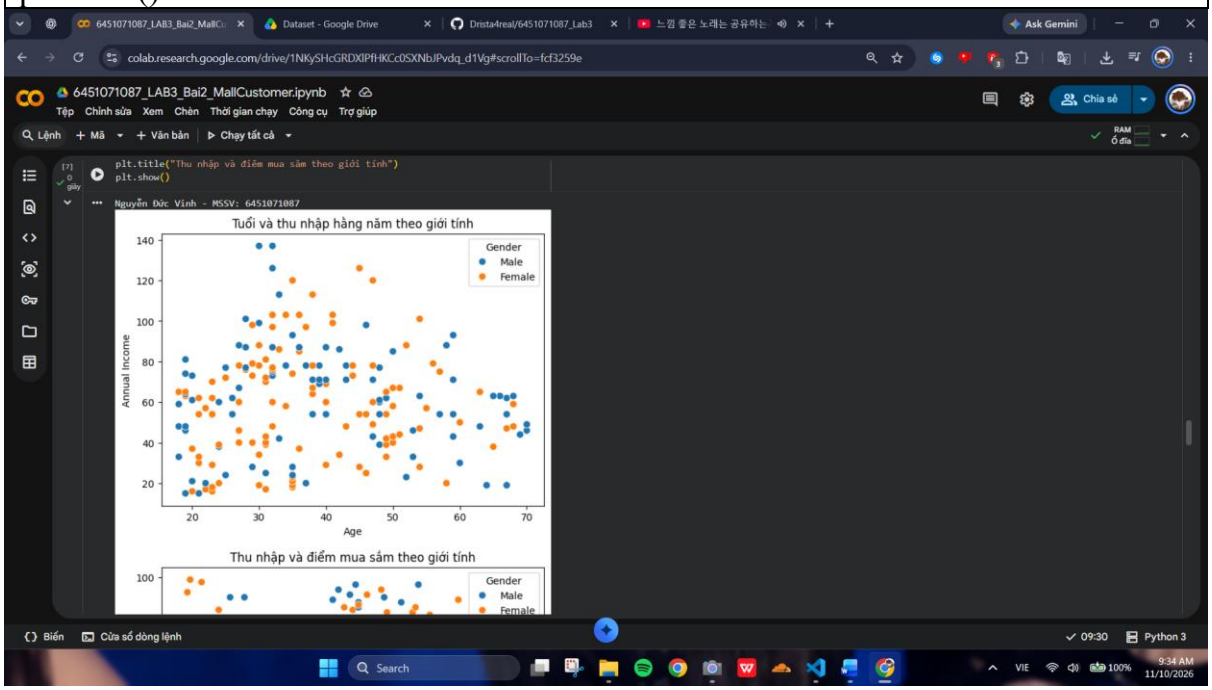
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
gender_counts = work["Gender"].value_counts()
print(gender_counts)
plt.figure(figsize=(6, 4))
gender_counts.plot(kind="bar", edgecolor="black")
plt.xlabel("Giới tính")
plt.ylabel("Số khách hàng")
plt.title("Số lượng khách hàng theo giới tính")
plt.xticks(rotation=0)
plt.show()
```

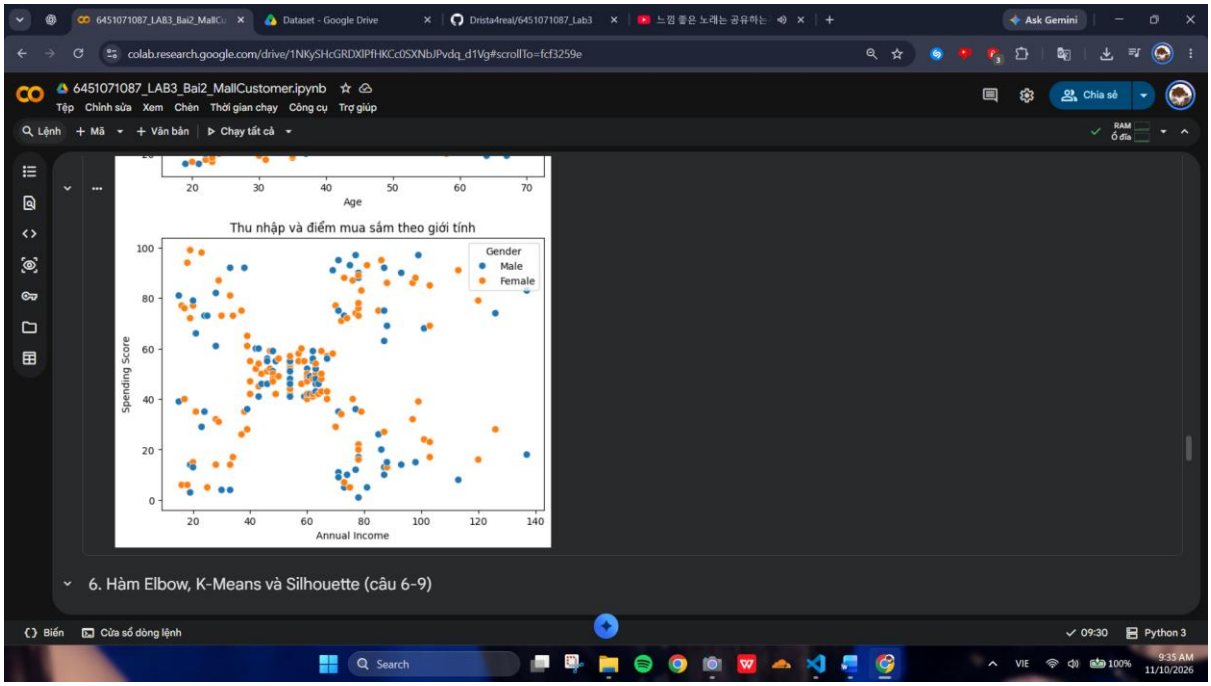


Scatter plot theo giới tính (câu 4, 5):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
plt.figure(figsize=(7, 5))
sns.scatterplot(data=work, x="Age", y="Annual Income", hue="Gender", s=55)
plt.title("Tuổi và thu nhập hằng năm theo giới tính")
plt.show()

plt.figure(figsize=(7, 5))
sns.scatterplot(data=work, x="Annual Income", y="Spending Score",
hue="Gender", s=55)
plt.title("Thu nhập và điểm mua sắm theo giới tính")
plt.show()
```





Hàm Elbow, K-Means và Silhouette (câu 6-9):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
def elbow_k(inertias):
    # Chọn điểm xa nhất so với đường thẳng nối k nhỏ nhất và k lớn nhất.
    x = np.arange(len(inertias), dtype=float)
    x_norm = (x - x.min()) / (x.max() - x.min())
    y = np.asarray(inertias, dtype=float)
    y_norm = (y - y.min()) / (y.max() - y.min() + 1e-12)
    distance = np.abs(x_norm + y_norm - 1) / np.sqrt(2)
    return int(np.argmax(distance) + 1)

def cluster_experiment(columns, title, plot_3d=False):
    X_original = work[columns].to_numpy(dtype=float)
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_original)

    k_values = range(1, 11)
    inertias = []
    for k in k_values:
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        km.fit(X_scaled)
        inertias.append(km.inertia_)

    plt.figure(figsize=(7, 4))
    plt.plot(list(k_values), inertias, marker="o")
    plt.xlabel("Số cụm k")
    plt.ylabel("Inertia (WSS)")
    plt.title(f"Elbow Method - {title}")
```

```
    plt.xticks(list(k_values))
    plt.grid(True)
    plt.show()

    best_k = elbow_k(inertias)
    print(f"Số cụm được chọn theo điểm khuỷu tay: k={best_k}")
    model = KMeans(n_clusters=best_k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels,
                             sample_size=min(2000, len(X_scaled)),
                             random_state=42) if 2 <= len(np.unique(labels)) < len(labels) else
np.nan
    print("Silhouette coefficient:", round(float(score), 4))

    result = work[columns].copy()
    result["Cluster"] = labels
    centers = scaler.inverse_transform(model.cluster_centers_)
    centers_df = pd.DataFrame(centers, columns=columns)
    centers_df["Cluster"] = np.arange(best_k)
    display(centers_df)

    if plot_3d:
        try:
            import plotly.express as px
        except ImportError:
            import subprocess, sys
            subprocess.check_call([sys.executable, "-m", "pip", "-q", "install", "plotly"])
            import plotly.express as px
        fig = px.scatter_3d(result, x=columns[0], y=columns[1], z=columns[2],
                            color=result["Cluster"].astype(str),
                            title=f"K-Means 3D - {title}")
        fig.show()
    else:
        plt.figure(figsize=(7, 5))
        sns.scatterplot(data=result, x=columns[0], y=columns[1],
                        hue="Cluster", palette="tab10", s=55)
        plt.scatter(centers[:, 0], centers[:, 1], marker="X", s=180,
                    edgecolor="black", label="Centroid")
        plt.title(f"K-Means clustering - {title}")
        plt.legend()
        plt.show()

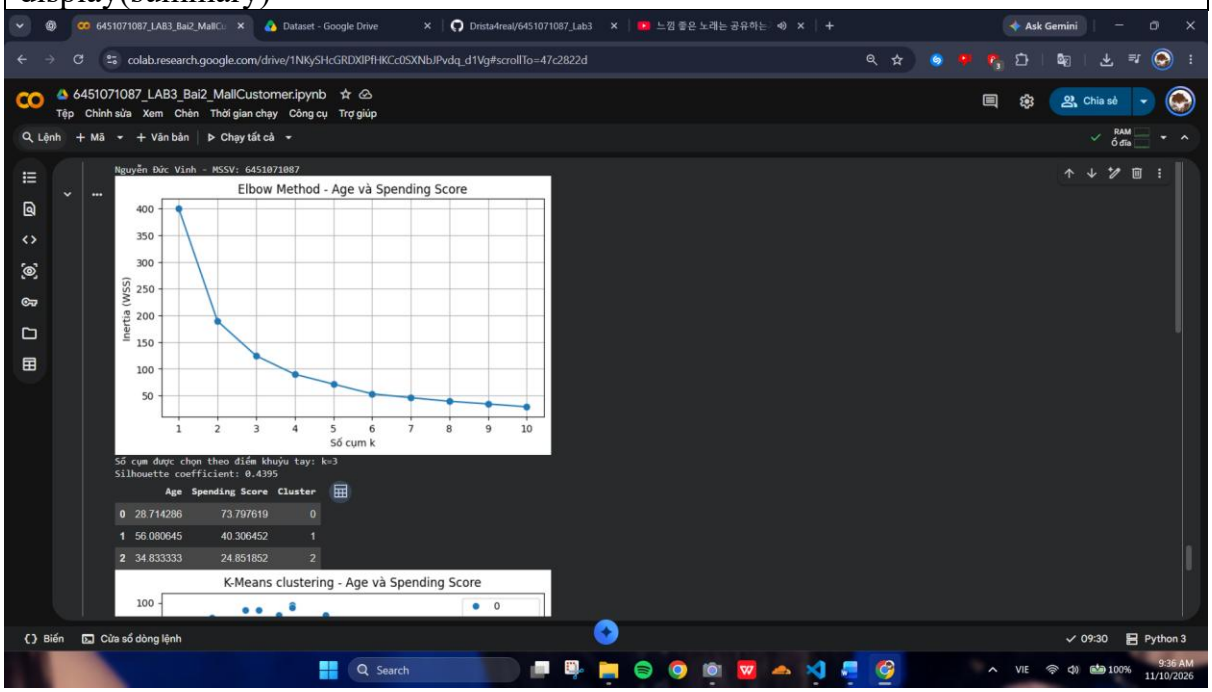
    print("Số khách hàng trong từng cụm:")
    display(result["Cluster"].value_counts().sort_index().to_frame("Count"))
    return model, scaler, result, best_k, score
```

```
# Câu 6-8: Tuổi và Spending Score
model_age_spending, scaler_age_spending, result_age_spending, k_age_spending,
sil_age_spending = cluster_experiment(
    ["Age", "Spending Score"], "Age và Spending Score"
)

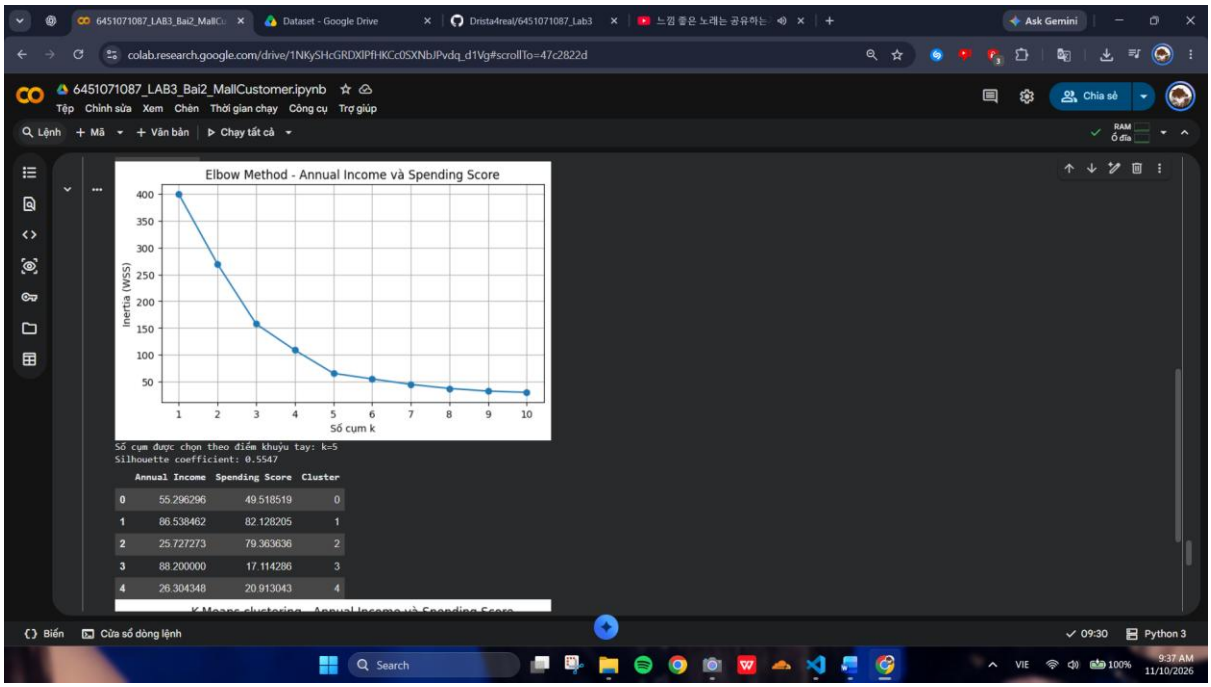
# Câu 9a: Thu nhập và Spending Score
model_income_spending, scaler_income_spending, result_income_spending,
k_income_spending, sil_income_spending = cluster_experiment(
    ["Annual Income", "Spending Score"], "Annual Income và Spending Score"
)

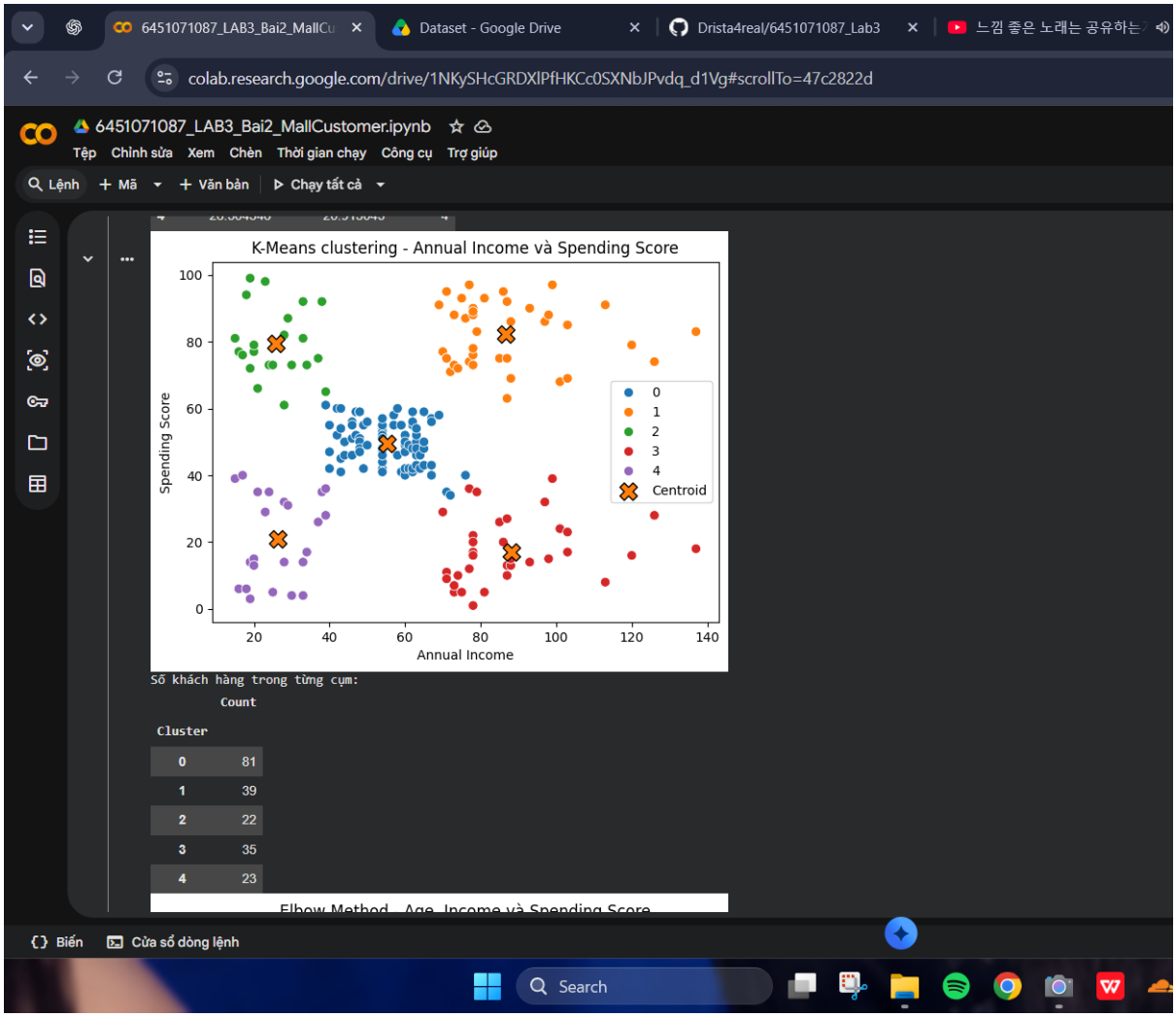
# Câu 9b: Tuổi, thu nhập và Spending Score
model_3d, scaler_3d, result_3d, k_3d, sil_3d = cluster_experiment(
    ["Age", "Annual Income", "Spending Score"], "Age, Income và Spending Score",
plot_3d=True
)

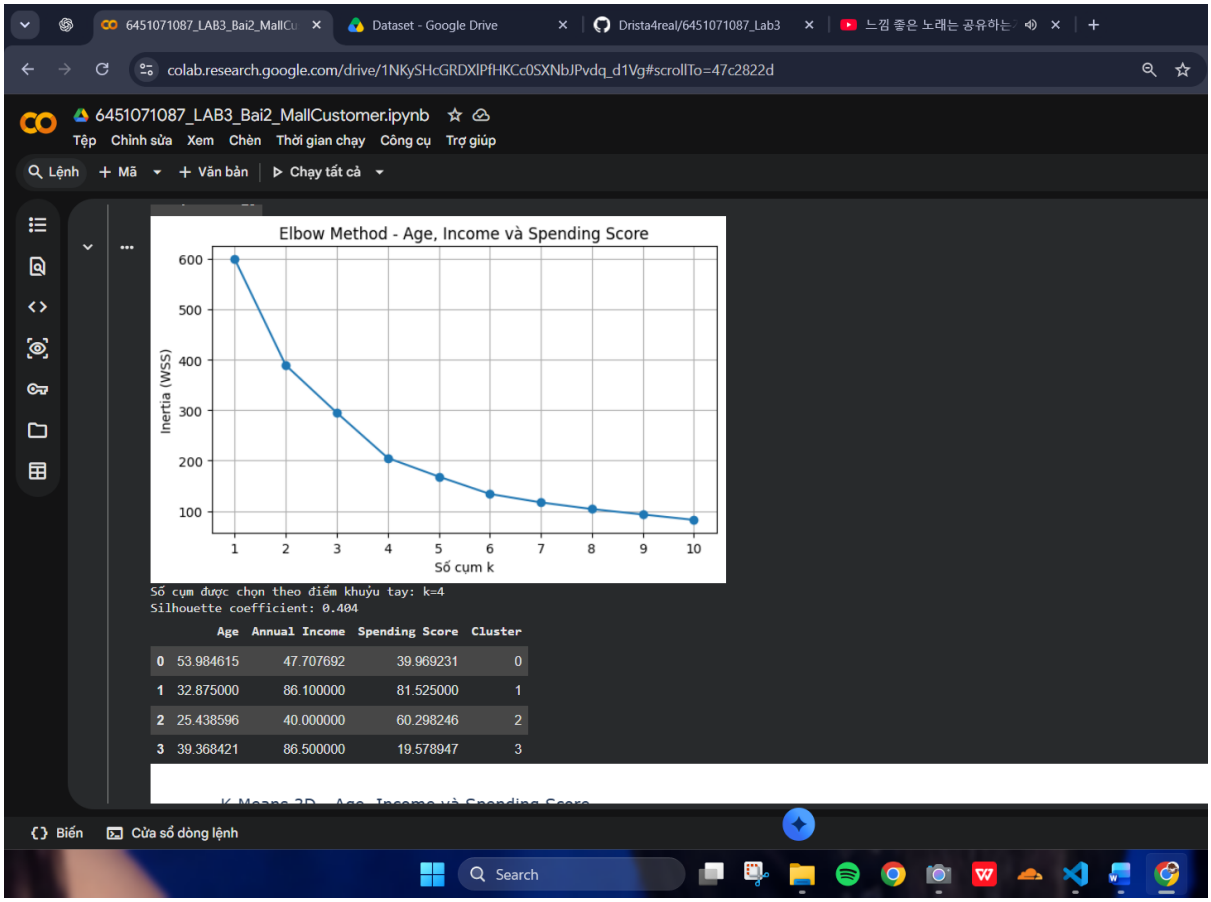
summary = pd.DataFrame({
    "Nhóm thuộc tính": ["Age + Spending Score", "Income + Spending Score", "Age
+ Income + Spending Score"],
    "k": [k_age_spending, k_income_spending, k_3d],
    "Silhouette": [sil_age_spending, sil_income_spending, sil_3d]
})
display(summary)
```

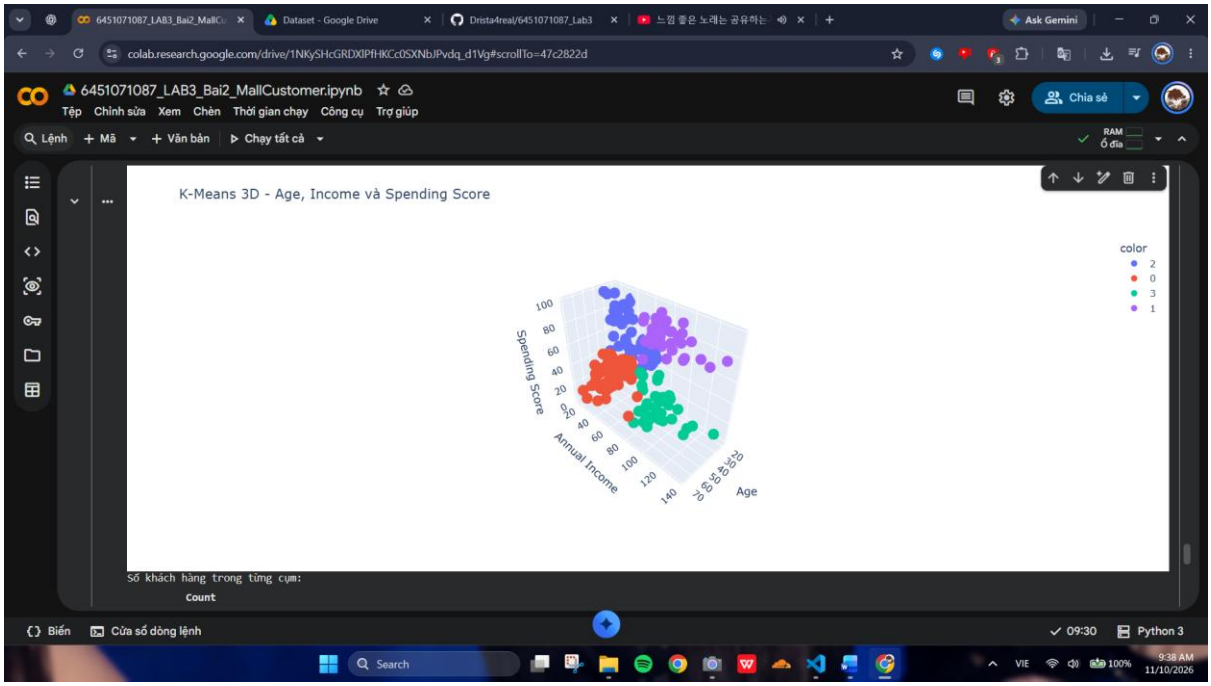


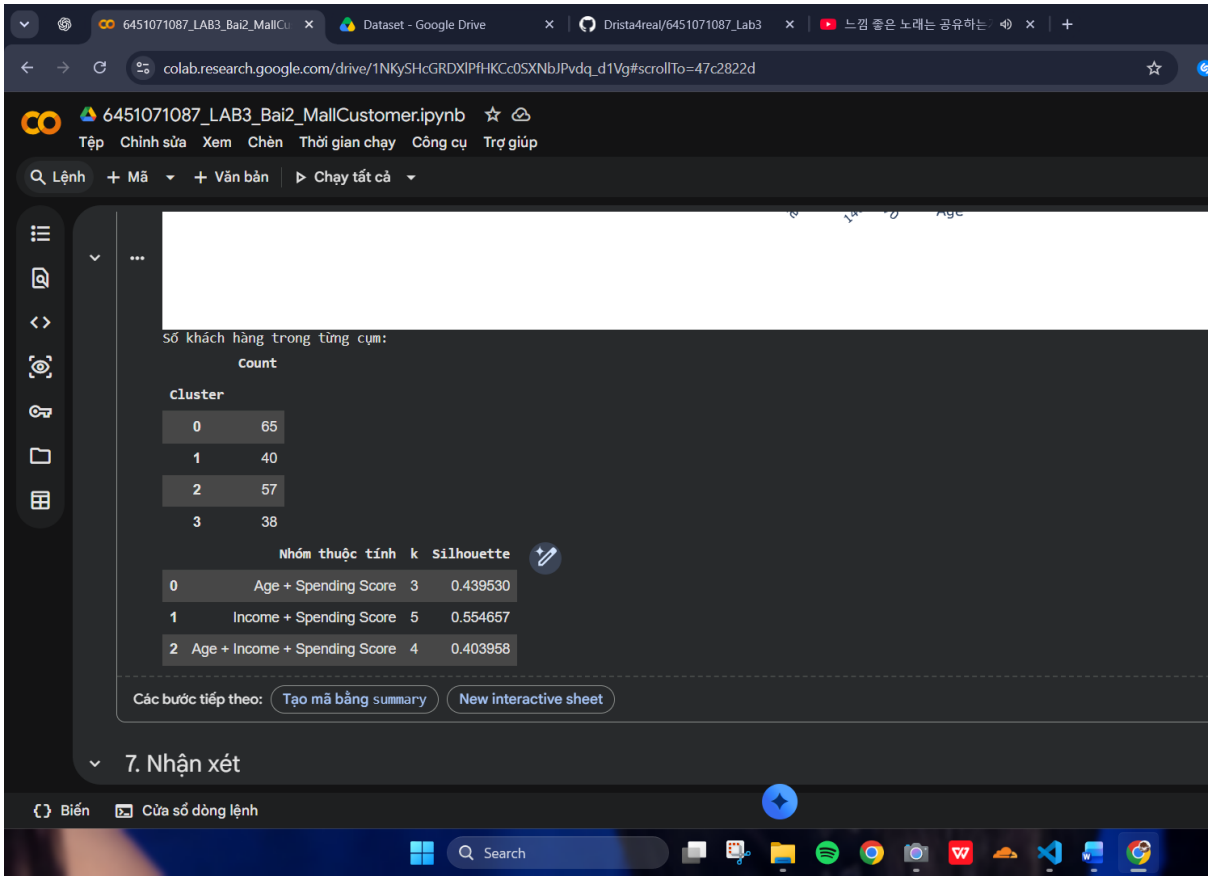

6451071087_LAB3_Bai2_MallCustomer.ipynb
K-Means clustering - Age và Spending Score
Spending Score
Age
0
1
2
Centroid
Số khách hàng trong từng cụm:
Count
Cluster
0 84
1 62
2 54


6451071087_LAB3_Bai2_MallCustomer.ipynb
Elbow Method - Annual Income và Spending Score
Inertia (WSS)
Số cụm k
Số cụm được chọn theo điểm khuỷu tay: k=5
Silhouette coefficient: 0.5547
Annual Income Spending Score Cluster
0 55.296296 49.518519 0
1 86.538462 82.128205 1
2 25.727273 79.363636 2
3 88.200000 17.114286 3
4 26.304348 20.913043 4

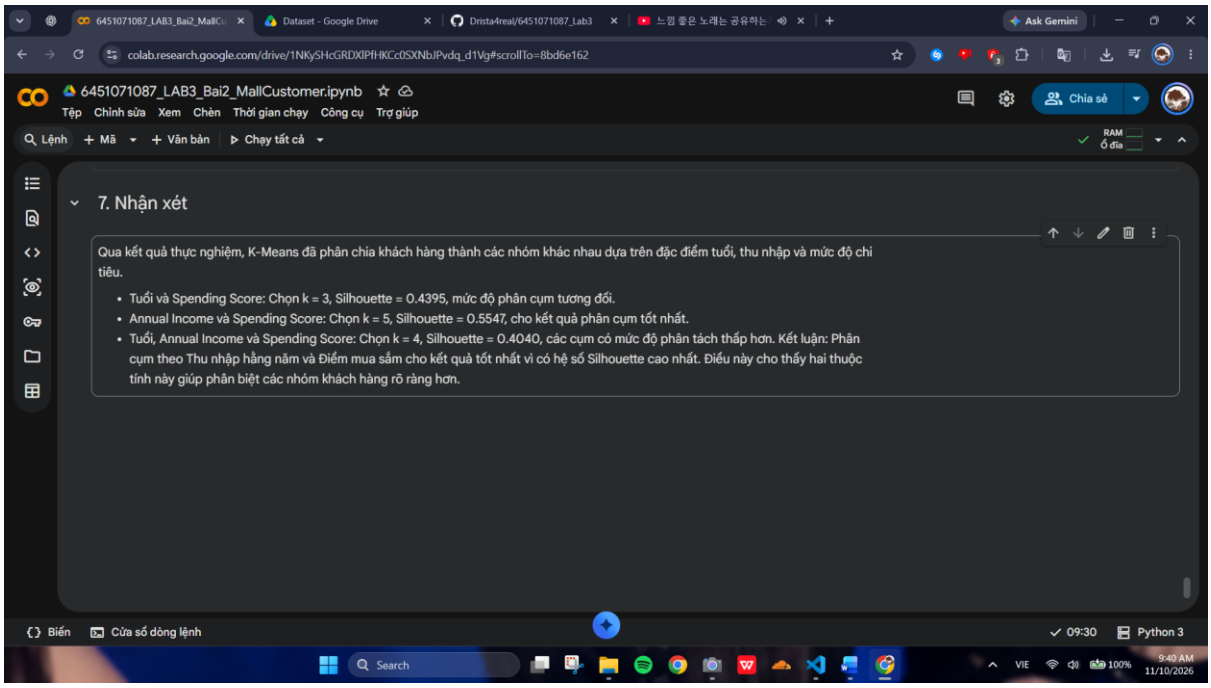

6451071087_LAB3_Bai2_MallCu
Dataset - Google Drive
Drista4real/6451071087_Lab3
colab.research.google.com/drive/1NKySHcGRDXlPfHKCc0SXNbJPvdq_d1Vg#scrollTo=47c2822d
6451071087_LAB3_Bai2_MallCustomer.ipynb
Tệp Chỉnh sửa Xem Chèn Thời gian chạy Công cụ Trợ giúp
Lệnh + Mã + Văn bản Chạy tất cả
K-Means clustering - Annual Income và Spending Score
Spending Score
Annual Income
0
1
2
3
4
Centroid
Số khách hàng trong từng cụm:
Count
Cluster
0 81
1 39
2 22
3 35
4 23
Elbow Method - Age, Income và Spending Score
Biến
Cửa sổ dòng lệnh
Search


6451071087_LAB3_Bai2_MallCustomer.ipynb
Tệp Chỉnh sửa Xem Chèn Thời gian chạy Công cụ Trợ giúp
Elbow Method - Age, Income và Spending Score
Inertia (WSS)
Số cụm k
Số cụm được chọn theo điểm khuỷu tay: k=4
Silhouette coefficient: 0.404
Age Annual Income Spending Score Cluster
0 53.984615 47.707692 39.969231 0
1 32.875000 86.100000 81.525000 1
2 25.438596 40.000000 60.298246 2
3 39.368421 86.500000 19.578947 3
Biến
Cửa sổ dòng lệnh


6451071087_LAB3_Bai2_MallCustomer.ipynb
Tệp Chỉnh sửa Xem Chèn Thời gian chạy Công cụ Trợ giúp
K-Means 3D - Age, Income và Spending Score
color
Spending Score
Annual Income
Age
Số khách hàng trong từng cụm:
Count
Biến
Cửa sổ dòng lệnh



Nhận xét:



Bài 5 (Trang 26) – Gom cụm MNIST bằng K-Means

Nạp MNIST từ Keras, chuyển mỗi ảnh 28x28 thành vector 784 chiều và chuẩn hóa pixel. Để giảm thời gian tính toán, notebook dùng mẫu phân tầng 10.000 ảnh từ tập train; số lượng mẫu được ghi rõ trong báo cáo.

Huấn luyện K-Means với 10 cụm, k-means++, n_init=10 và random_state=42. Do mã cụm không tương ứng trực tiếp với nhãn số, dùng Hungarian algorithm để ánh xạ mã cụm về chữ số trước khi tính các độ đo.

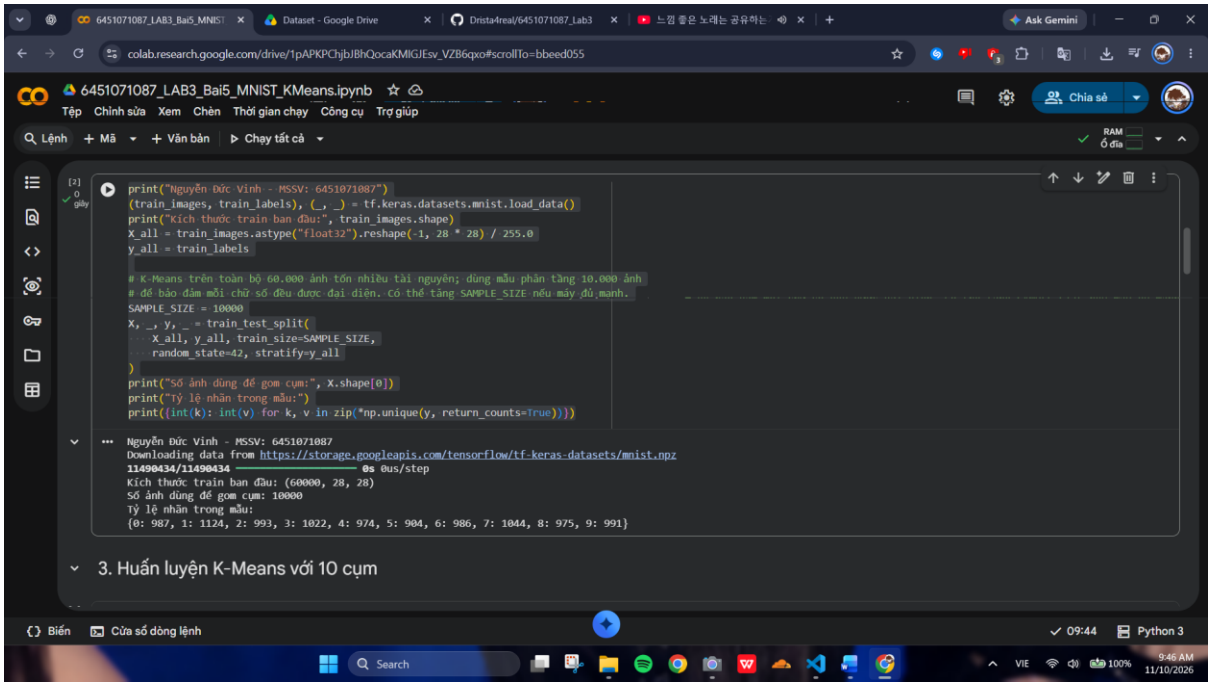
Đánh giá bằng confusion matrix, Accuracy, Precision, Recall, F1-score; hiển thị tâm 10 cụm và một số ảnh mẫu.

Mã nguồn: Notebook: 6451071087_LAB3_Bai5_MNIST_KMeans.ipynb

Đọc MNIST, chuẩn hóa và lấy mẫu phân tầng:

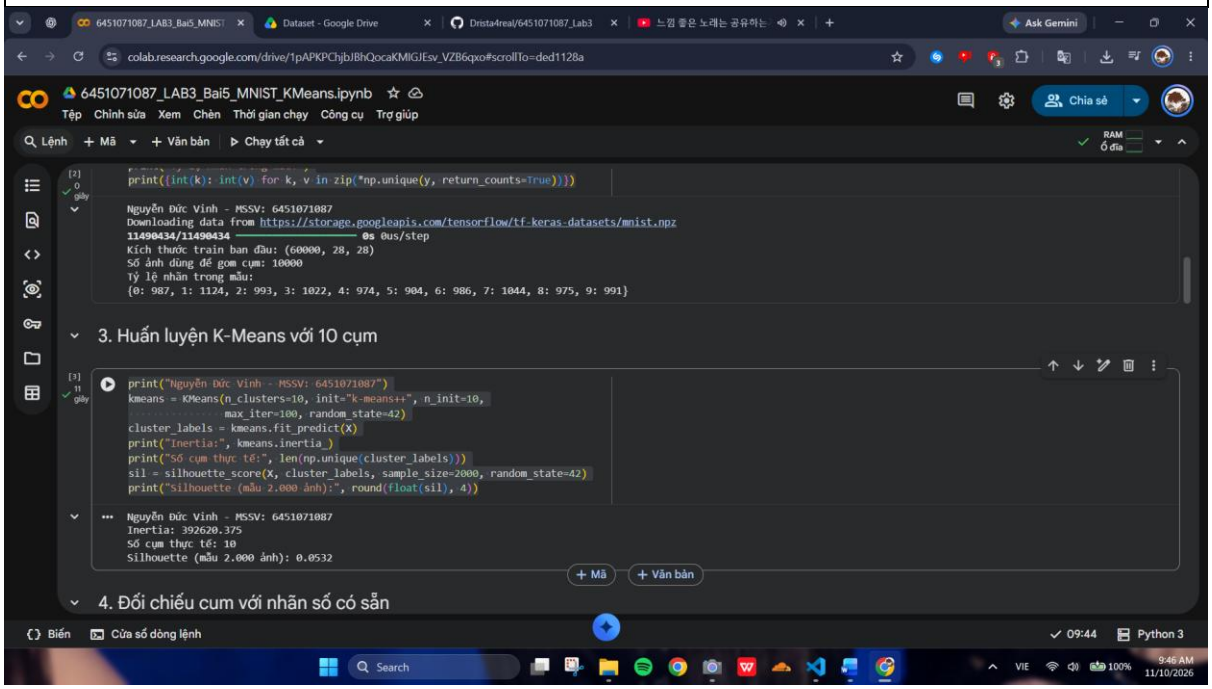
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
(train_images, train_labels), (_, _) = tf.keras.datasets.mnist.load_data()
print("Kích thước train ban đầu:", train_images.shape)
X_all = train_images.astype("float32").reshape(-1, 28 * 28) / 255.0
y_all = train_labels


# K-Means trên toàn bộ 60.000 ảnh tốn nhiều tài nguyên; dùng mẫu phân tầng 10.000 ảnh
# để bảo đảm mỗi chữ số đều được đại diện. Có thể tăng SAMPLE_SIZE nếu máy đủ mạnh.
SAMPLE_SIZE = 10000
X, _, y, _ = train_test_split(
    X_all, y_all, train_size=SAMPLE_SIZE,
    random_state=42, stratify=y_all
)
print("Số ảnh dùng để gom cụm:", X.shape[0])
print("Tỷ lệ nhãn trong mẫu:")
print({int(k): int(v) for k, v in zip(*np.unique(y, return_counts=True))})
```



Huấn luyện K-Means với 10 cụm:

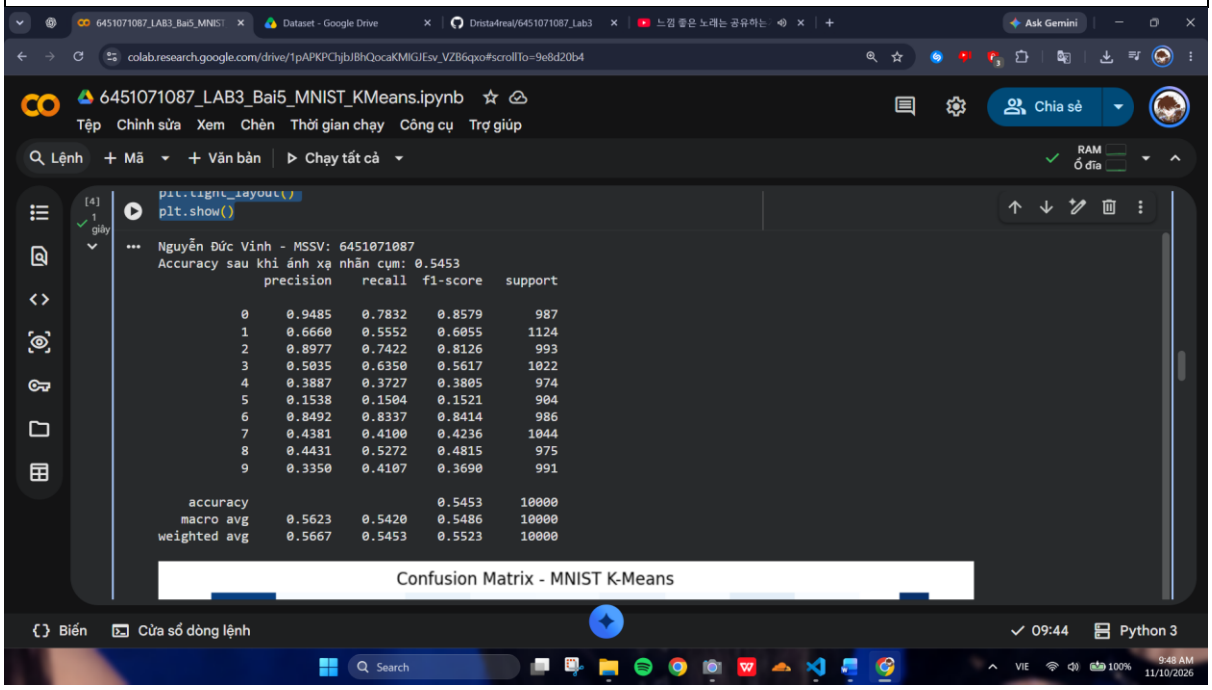
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
kmeans = KMeans(n_clusters=10, init="k-means++", n_init=10,
                max_iter=100, random_state=42)
cluster_labels = kmeans.fit_predict(X)
print("Inertia:", kmeans.inertia_)
print("Số cụm thực tế:", len(np.unique(cluster_labels)))
sil = silhouette_score(X, cluster_labels, sample_size=2000, random_state=42)
print("Silhouette (mẫu 2.000 ảnh):", round(float(sil), 4))
```

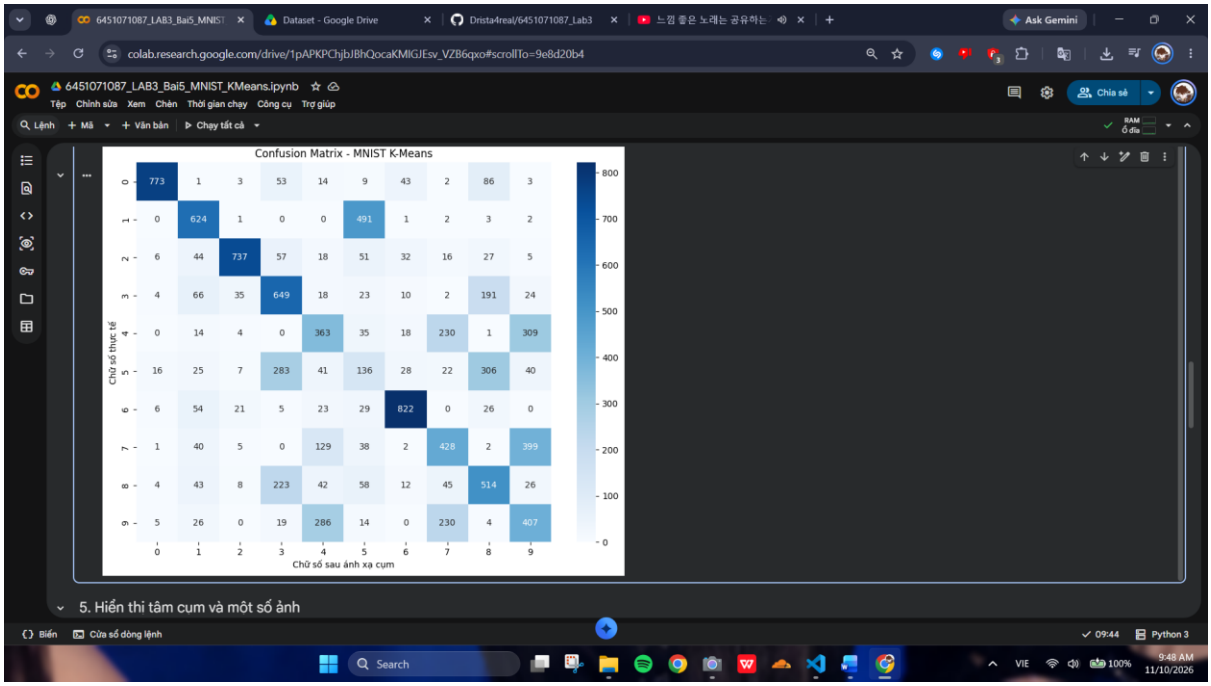


Đối chiếu cụm với nhãn số có sẵn:

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
# Nhãn cụm K-Means chỉ là mã cụm; dùng Hungarian algorithm để ánh xạ tối ưu
# các mã cụm về chữ số thật trước khi lập confusion matrix.
contingency = confusion_matrix(y, cluster_labels, labels=np.arange(10))
row_ind, col_ind = linear_sum_assignment(-contingency)
cluster_to_digit = {int(cluster): int(digit) for digit, cluster in zip(row_ind, col_ind)}
mapped_predictions = np.array([cluster_to_digit[int(c)] for c in cluster_labels])

cm = confusion_matrix(y, mapped_predictions, labels=np.arange(10))
print("Accuracy sau khi ánh xạ nhãn cụm:", round(accuracy_score(y,
mapped_predictions), 4))
print(classification_report(y, mapped_predictions, labels=np.arange(10),
                            digits=4, zero_division=0))
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=np.arange(10), yticklabels=np.arange(10))
plt.xlabel("Chữ số sau ánh xạ cụm")
plt.ylabel("Chữ số thực tế")
plt.title("Confusion Matrix - MNIST K-Means")
plt.tight_layout()
plt.show()
```

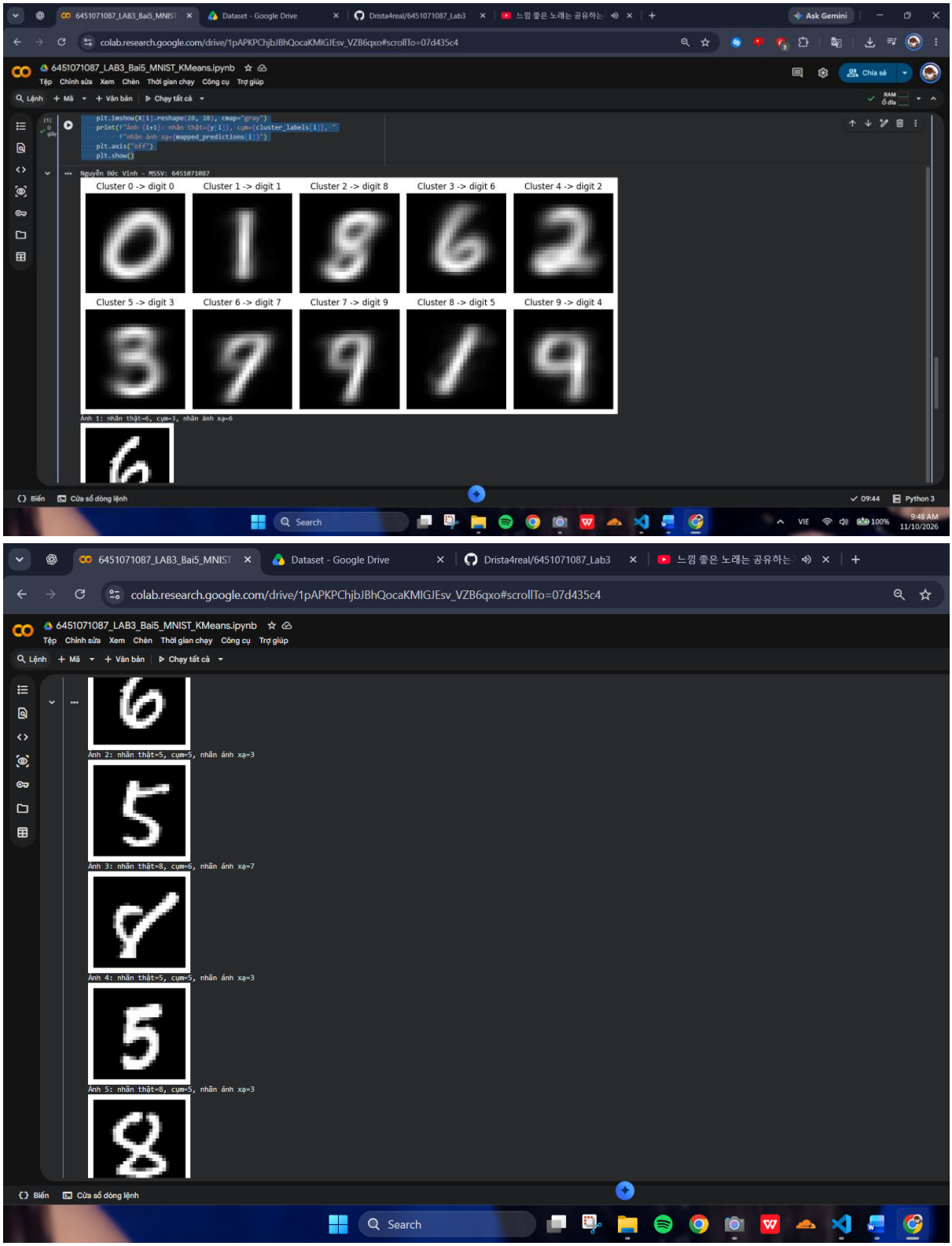




Hiển thị tâm cụm và một số ảnh:

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
fig, axes = plt.subplots(2, 5, figsize=(11, 5))
for cluster_id, ax in enumerate(axes.ravel()):
    ax.imshow(kmeans.cluster_centers_[cluster_id].reshape(28, 28), cmap="gray")
    ax.set_title(f"Cluster {cluster_id} -> digit {cluster_to_digit[cluster_id]}")
    ax.axis("off")
plt.tight_layout()
plt.show()

for i in range(5):
    plt.figure(figsize=(2.2, 2.2))
    plt.imshow(X[i].reshape(28, 28), cmap="gray")
    print(f"Ảnh {i+1}: nhãn thật={y[i]}, cụm={cluster_labels[i]}, "
          f"nhãn ánh xạ={mapped_predictions[i]}")
    plt.axis("off")
    plt.show()
```



K-Means là thuật toán không giám sát nên mã cụm không mặc định trùng với chữ số. Vì vậy cần ánh xạ cụm sang chữ số trước khi tính confusion matrix và các chỉ số phân lớp. Thí nghiệm sử dụng mẫu phân tầng 10.000 ảnh từ tập train 60.000 ảnh để giảm chi phí tính toán

Bài 6(Trang 26) – Giảm số màu ảnh

Đọc một ảnh RGB, chuyển giá trị màu từ [0,255] về [0,1], chuyển ảnh từ mảng 3 chiều thành ma trận pixel 2 chiều. Notebook tìm ảnh trong thư mục Drive đã cung cấp; nếu có nhiều ảnh, cần chọn đúng file bằng biến IMAGE_PATH.

Áp dụng K-Means với 32 cụm màu. Với ảnh lớn, mô hình fit trên tối đa 50.000 pixel mẫu rồi dự đoán nhãn cho toàn bộ pixel. Tâm cụm được dùng để tái tạo ảnh RGB.

Hiển thị ảnh gốc và ảnh sau lượng tử hóa màu; lưu ảnh kết quả; so sánh dung lượng file. Dung lượng sau lưu phụ thuộc định dạng nén nên không khẳng định luôn nhỏ hơn ảnh gốc.

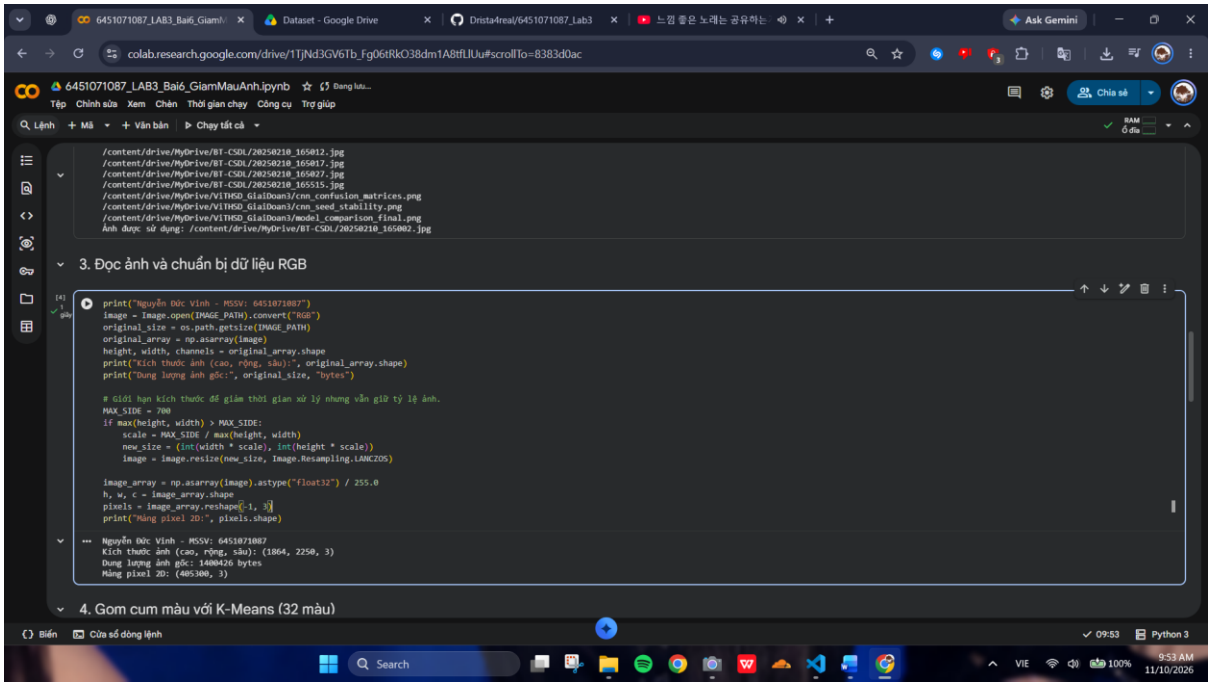
Mã nguồn: Notebook: 6451071087_LAB3_Bai6_GiamMauAnh.ipynb

Đọc ảnh và chuẩn bị dữ liệu RGB:

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
image = Image.open(IMAGE_PATH).convert("RGB")
original_size = os.path.getsize(IMAGE_PATH)
original_array = np.asarray(image)
height, width, channels = original_array.shape
print("Kích thước ảnh (cao, rộng, sâu):", original_array.shape)
print("Dung lượng ảnh gốc:", original_size, "bytes")

# Giới hạn kích thước để giảm thời gian xử lý nhưng vẫn giữ tỷ lệ ảnh.
MAX_SIDE = 700
if max(height, width) > MAX_SIDE:
    scale = MAX_SIDE / max(height, width)
    new_size = (int(width * scale), int(height * scale))
    image = image.resize(new_size, Image.Resampling.LANCZOS)

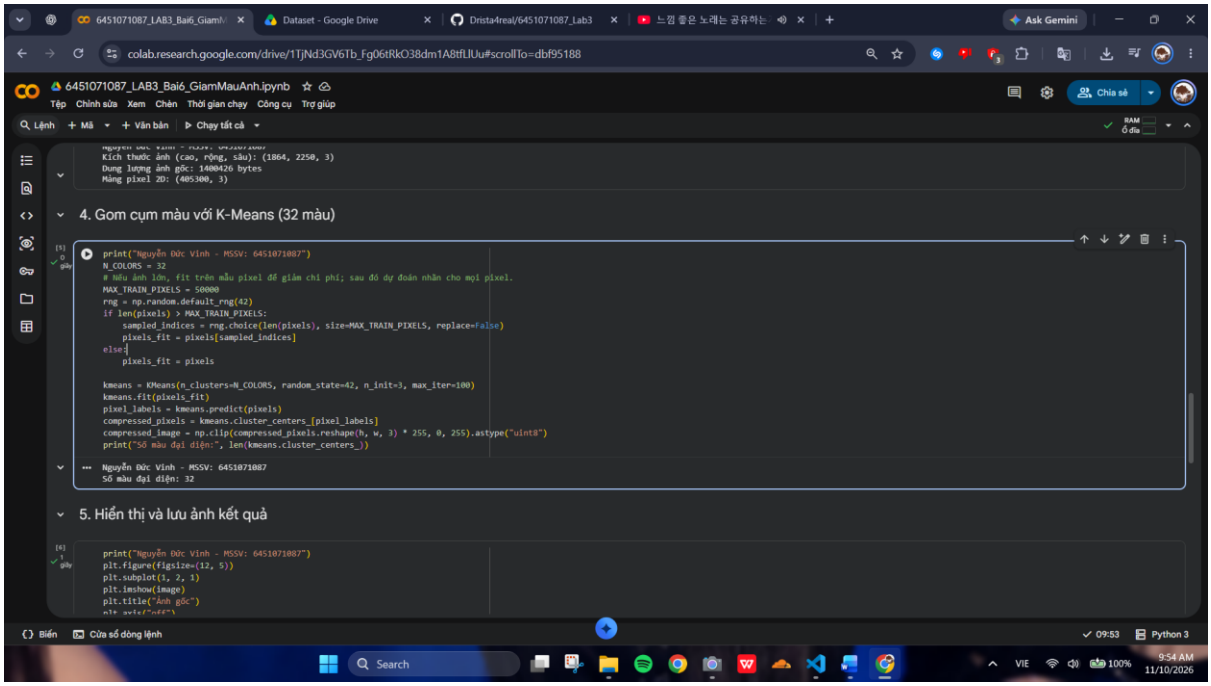
image_array = np.asarray(image).astype("float32") / 255.0
h, w, c = image_array.shape
pixels = image_array.reshape(-1, 3)
print("Mảng pixel 2D:", pixels.shape)
```



Gom cụm màu với K-Means (32 màu):

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
N_COLORS = 32
# Nếu ảnh lớn, fit trên mẫu pixel để giảm chi phí; sau đó dự đoán nhãn cho mọi
pixel.
MAX_TRAIN_PIXELS = 50000
rng = np.random.default_rng(42)
if len(pixels) > MAX_TRAIN_PIXELS:
    sampled_indices = rng.choice(len(pixels), size=MAX_TRAIN_PIXELS,
replace=False)
    pixels_fit = pixels[sampled_indices]
else:
    pixels_fit = pixels

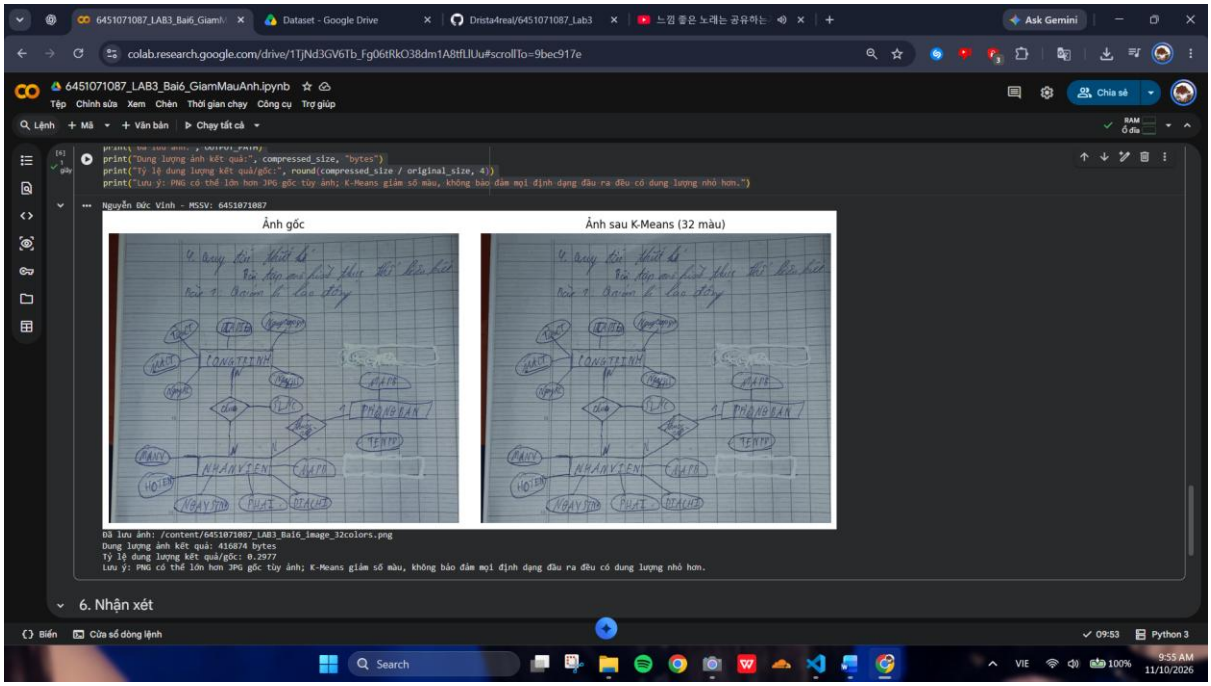
kmeans = KMeans(n_clusters=N_COLORS, random_state=42, n_init=3,
max_iter=100)
kmeans.fit(pixels_fit)
pixel_labels = kmeans.predict(pixels)
compressed_pixels = kmeans.cluster_centers_[pixel_labels]
compressed_image = np.clip(compressed_pixels.reshape(h, w, 3) * 255, 0,
255).astype("uint8")
print("Số màu đại diện:", len(kmeans.cluster_centers_))
```



Hiển thị và lưu ảnh kết quả:

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Ảnh gốc")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(compressed_image)
plt.title(f"Ảnh sau K-Means ({N_COLORS} màu)")
plt.axis("off")
plt.tight_layout()
plt.show()

OUTPUT_PATH = "/content/6451071087_LAB3_Bai6_image_32colors.png"
Image.fromarray(compressed_image).save(OUTPUT_PATH, optimize=True)
compressed_size = os.path.getsize(OUTPUT_PATH)
print("Đã lưu ảnh:", OUTPUT_PATH)
print("Dung lượng ảnh kết quả:", compressed_size, "bytes")
print("Tỷ lệ dung lượng kết quả/gốc:", round(compressed_size / original_size, 4))
print("Lưu ý: PNG có thể lớn hơn JPG gốc tùy ảnh; K-Means giảm số màu, không bảo đảm mọi định dạng đầu ra đều có dung lượng nhỏ hơn.")
```



K-Means gom các pixel có màu gần nhau vào 32 cụm và thay mỗi pixel bằng tâm cụm tương ứng. Việc này giảm số màu xuất hiện trong ảnh. Dung lượng file phụ thuộc định dạng, bộ mã hóa và nội dung ảnh; do đó cần so sánh kích thước file thực tế thay vì khẳng định luôn giảm.

Bài 5(Trang 29)

Nạp bộ Breast Cancer Wisconsin (Diagnostic) từ sklearn. Dữ liệu có 30 thuộc tính số và hai nhãn benign/malignant. Chuẩn hóa thuộc tính bằng StandardScaler.

Huấn luyện K-Means với k=2 và k-means++. Không sử dụng nhãn thực tế khi huấn luyện. Sau khi gom cụm, dùng Hungarian algorithm để ánh xạ mã cụm sang nhãn benign/malignant phục vụ đánh giá.

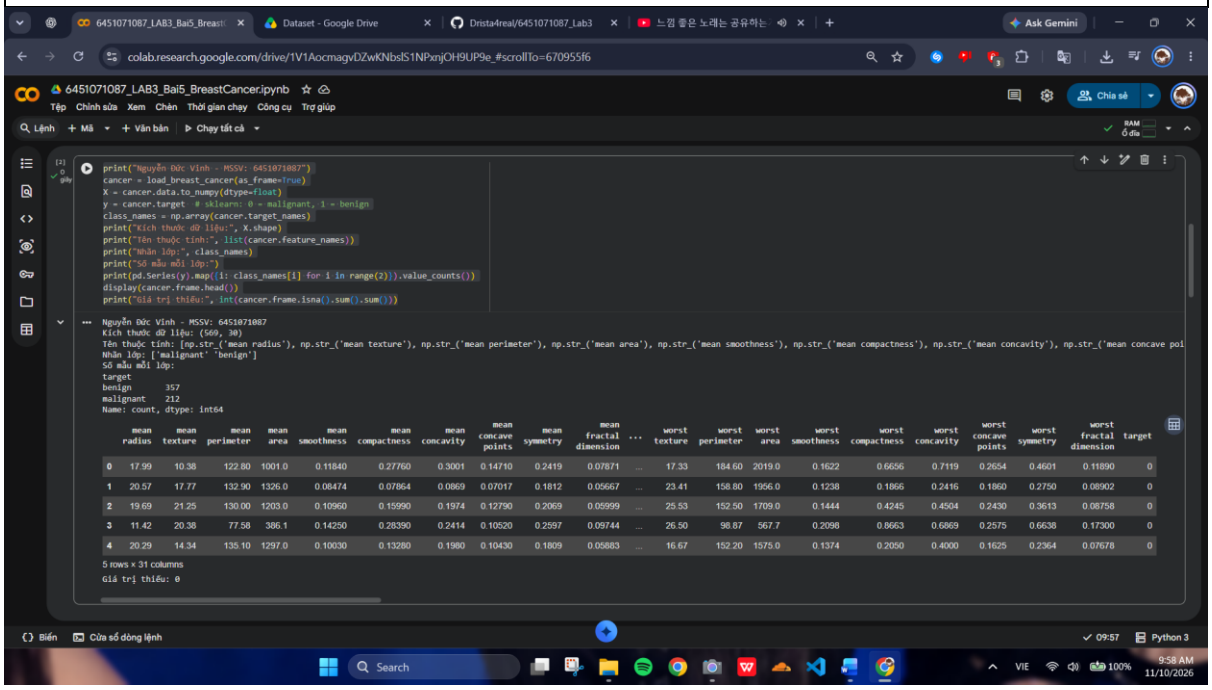
Đánh giá bằng confusion matrix, Accuracy, Precision, Recall, F1-score và Silhouette coefficient; dùng PCA 2 chiều để trực quan hóa kết quả.

Mã nguồn: Notebook: 6451071087_LAB3_Bai5_BreastCancer.ipynb

Đọc bộ dữ liệu Breast Cancer Wisconsin (Diagnostic)

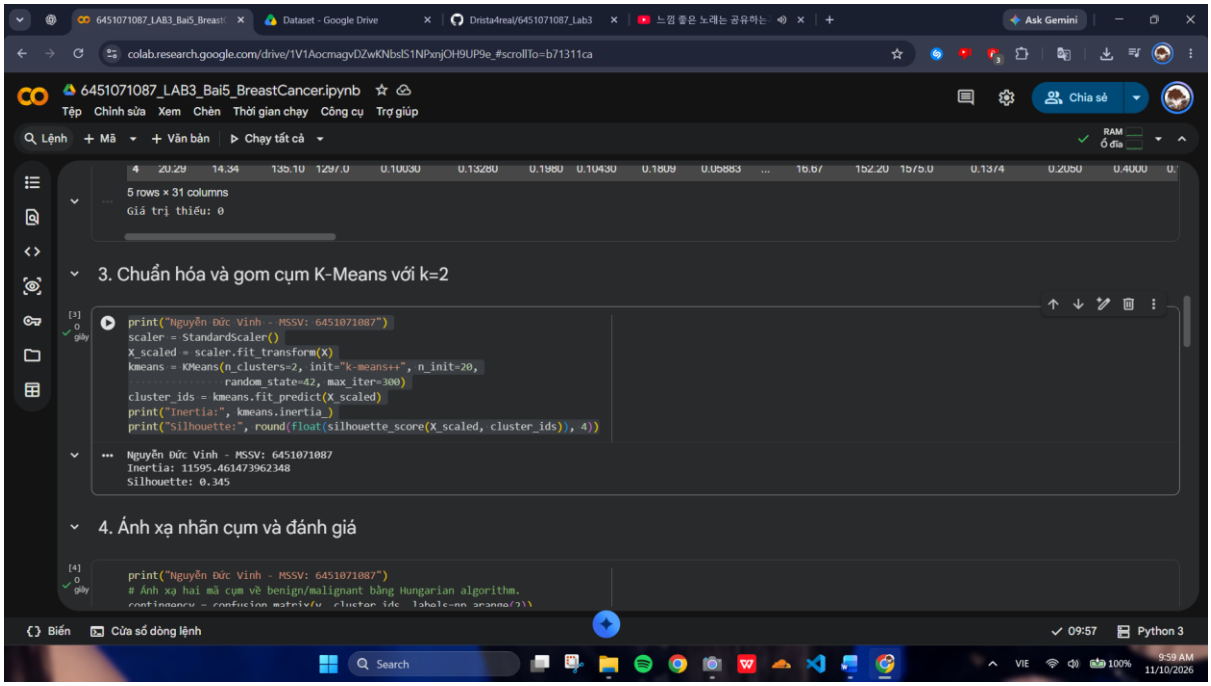
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
cancer = load_breast_cancer(as_frame=True)
X = cancer.data.to_numpy(dtype=float)
y = cancer.target  # sklearn: 0 = malignant, 1 = benign
class_names = np.array(cancer.target_names)
print("Kích thước dữ liệu:", X.shape)
print("Tên thuộc tính:", list(cancer.feature_names))
print("Nhãn lớp:", class_names)
print("Số mẫu mỗi lớp:")
```

```
print(pd.Series(y).map({i: class_names[i] for i in range(2)}).value_counts())
display(cancer.frame.head())
print("Giá trị thiếu:", int(cancer.frame.isna().sum().sum()))
```



Chuẩn hóa và gom cụm K-Means với k=2

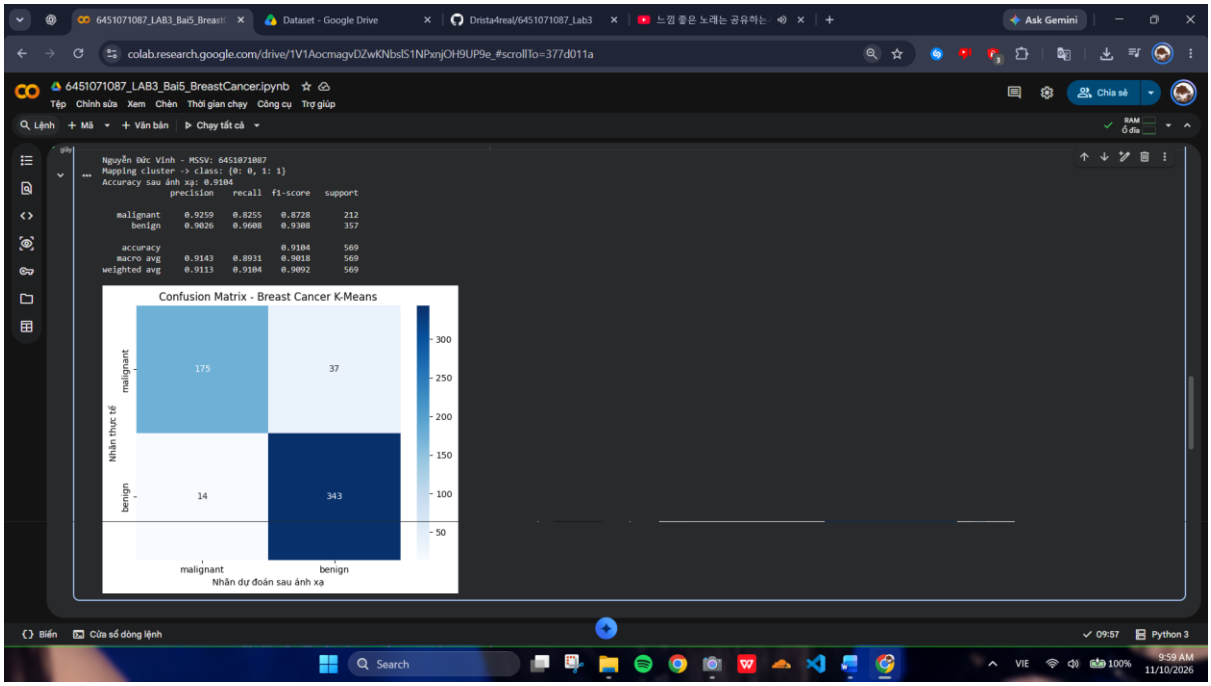
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
kmeans = KMeans(n_clusters=2, init="k-means++", n_init=20,
                random_state=42, max_iter=300)
cluster_ids = kmeans.fit_predict(X_scaled)
print("Inertia:", kmeans.inertia_)
print("Silhouette:", round(float(silhouette_score(X_scaled, cluster_ids)), 4))
```



Ánh xạ nhãn cụm và đánh giá:

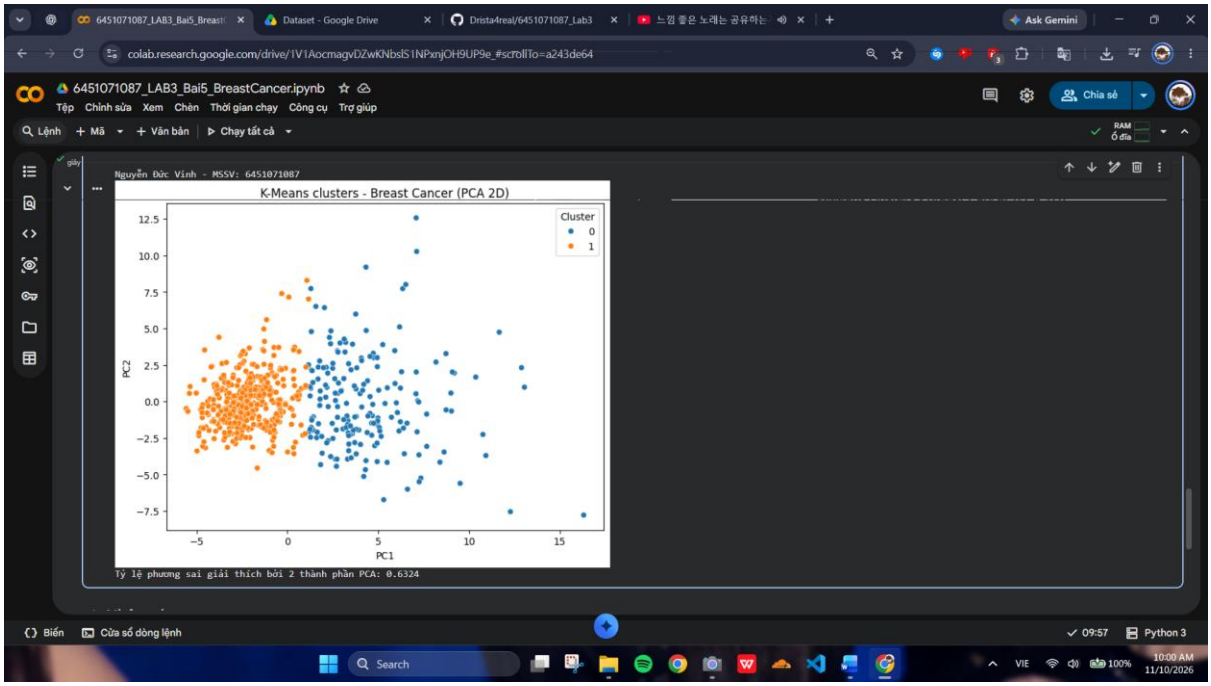
```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
# Ánh xạ hai mã cụm về benign/malignant bằng Hungarian algorithm.
contingency = confusion_matrix(y, cluster_ids, labels=np.arange(2))
row_ind, col_ind = linear_sum_assignment(-contingency)
cluster_to_class = {int(cluster): int(actual) for actual, cluster in zip(row_ind,
col_ind)}
predicted_class = np.array([cluster_to_class[int(c)] for c in cluster_ids])

cm = confusion_matrix(y, predicted_class, labels=np.arange(2))
print("Mapping cluster -> class:", cluster_to_class)
print("Accuracy sau ánh xạ:", round(accuracy_score(y, predicted_class), 4))
print(classification_report(y, predicted_class, labels=np.arange(2),
                            target_names=class_names, digits=4, zero_division=0))
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Nhãn dự đoán sau ánh xạ")
plt.ylabel("Nhãn thực tế")
plt.title("Confusion Matrix - Breast Cancer K-Means")
plt.tight_layout()
plt.show()
```



Trực quan hóa kết quả bằng PCA:

```
print("Nguyễn Đức Vinh - MSSV: 6451071087")
pca = PCA(n_components=2, random_state=42)
X_2d = pca.fit_transform(X_scaled)
plot_df = pd.DataFrame({
    "PC1": X_2d[:, 0],
    "PC2": X_2d[:, 1],
    "Cluster": cluster_ids.astype(str),
    "Actual class": [class_names[int(v)] for v in y]
})
plt.figure(figsize=(8, 6))
sns.scatterplot(data=plot_df, x="PC1", y="PC2", hue="Cluster", s=35)
plt.title("K-Means clusters - Breast Cancer (PCA 2D)")
plt.show()
print("Tỷ lệ phương sai giải thích bởi 2 thành phần PCA:",
```



K-Means không sử dụng nhãn benign/malignant khi huấn luyện. Nhãn thật chỉ được dùng sau đó để ánh xạ mã cụm và đánh giá. Các chỉ số phân lớp sau ánh xạ giúp định lượng mức tương đồng, nhưng không biến mô hình gom cụm thành công cụ chẩn đoán y tế; kết quả chỉ phục vụ mục đích học thuật.

--- HẾT---